

混合精度算法的数值方法综述

by the ECP Multiprecision Effort Team (负责人: Hartwig Anzt)

目 录

1	简介	1
2	稠密线性代数	2
§2.1	低精度BLAS	2
§2.1.1	半精度硬件加速	2
§2.1.2	半精度GEMM (HGEMM)	4
§2.1.3	批量HGEMM	5
§2.2	经典迭代细化	5
§2.3	GMRES-IR	8
§2.3.1	缩放	9
§2.4	混合精度分解	10
§2.5	Cholesky 分解	10
§2.5.1	缩放	12
§2.5.2	移位	12
§2.6	最小二乘问题的迭代细化	13
§2.6.1	基于Cholesky 的方法	13
§2.6.2	增广矩阵方法	15
§2.7	量化整数LU 分解	13
§2.7.1	量化整数LU算法	16
§2.7.2	量化整数LU数值结果	17
§2.7.3	量化整数LU的未来工作	17
§2.8	对称特征值问题	18

3	多精度算法的数据和通信压缩	21
§3.1	数据转换	22
§3.2	数据压缩	23
§3.2.1	混合精度MPI	24
§3.3	近似快速傅里叶变换	25
§3.3.1	具有精度-速度折衷的近似FFT	25
§3.3.2	精度控制	26
§3.3.3	动态拆分	26
4	多精度稀疏分解	26
§4.1	多精度稀疏LU 和QR	26
§4.2	多精度稀疏直接求解器	27
5	Krylov 求解器技术的多精度工作	28
§5.1	兰佐斯CG	28
§5.1.1	理论结果	??
§5.1.2	实际应用	29
§5.2	阿诺尔迪-QR MGS-GMRES	29
§5.3	替代方法	31
6	多精度预处理器	32
7	将算术格式与内存格式分离的多精度工作	34
§7.1	在多重网格方法中使用不同的精度格式CG	34
8	在多重网格方法中使用不同的精度格式	38
9	xSDK数学库的多精度功能和互操作性	39
§9.1	Ginkgo	39
§9.2	heFFTe	39
§9.3	hypre CG	40

§9.4	Kokkos Kernels CG	40
§9.5	MAGMA CG	40
§9.6	PETSc CG	41
§9.7	PETSc CG	41
§9.8	SLATE CG	42
§9.9	STRUMPACK CG	42
§9.10	SuperLU CG	42
§9.11	Trilinos CG	43
10	IEEE格式和格式转换	43
§10.1	仿真器	43
§10.2	舍入误差分析	43

1 简介

在过去的几年里，硬件供应商已经开始设计低精度的特殊功能单元，以响应机器学习社区的需求以及他们对低精度格式的高计算能力的需求。此外，服务器也越来越多地采用低精度特殊功能单元，例如，美国橡树岭国家实验室(Oak Ridge National Laboratory)的超级计算机 Summit¹中的 NVIDIA²张量核心提供了比 IEEE 双精度计算机高出一个数量级的性能。与此同时，计算能力与存储器带宽³之间的差距的不断扩大，使得算术运算的成本比数据访问和通信成本低得多。美国在研E级计算计划(Exascale Computing Project) 在忽视硬件趋势和继续传统路径之间作出选择，并根据不断变化的硬件设计调整软件栈，因此决定采取积极步骤，开展一个多精度聚焦工作，以应对设计和工程新算法的挑战，利用低精度的计算能力，并调整通信格式，以满足应用程序的特定需求。为了启动多精度聚焦工作，我们在数值线性代数方面做了一些工作，在这篇综述里总结了所有现有的多精度知识，专业知识和软件功能。此外，这篇综述还包括目前的成果和一些初步结果，这些初步结果可能还没有被认为是“成熟的技术”，但有希望成长为多精度聚焦工作的生产质量。由于我们希望读者熟悉数值线性代数的基础知识，所以我们不提供算法本身的详细背景知识，而是集中讨论混合精度和多精度技术如何能够帮助改善这些方法的性能，并展示了应用的亮点，这些应用明显优于传统的固定精度方法。

¹美国超级计算机 Summit 位于美国田纳西州橡树岭国家实验室(ORNL)，由 IBM 公司在2018年制造，Summit 峰值速度可达 148600 Tflop/s (每秒万亿次浮点运算)，如今在全球超级计算机排行榜中排第二，第一是日本的超级计算机 Fugaku，峰值速度达到 442010 Tflop/s。

²NVIDIA 译作英伟达，是一家人工智能计算公司。公司创立于1993年，总部位于美国加利福尼亚州圣克拉拉市，专注于打造能够增强个人和专业计算平台的人机交互体验的产品，是全球可编程图形处理技术领袖。

³存储器带宽(memory bandwidth)，是指单位时间里存储器所存取的信息量，也称为存储器在单位时间内读出/写入的位数或字节数。

2 稠密线性代数

2.1 低精度BLAS

机器学习应用程序和人工智能(AI)的革命激起了人们对开发高性能半精度运算(16位浮点格式)的兴趣,因为大多数AI 应用程序不一定需要单精度或双精度的精确度[1]。半精度还使机器学习应用程序运行得更快,这不仅是因为运算速度更快,而且还因为与单精度相比,内存存储和流量减少了2倍,与双精度相比减少了4倍。

在供应商支持方面,NVIDIA、Google 和AMD 制造的硬件能够使用16位格式执行浮点运算。谷歌的Tensor Processing Units (TPUs) 是定制的芯片,主要是为使用bfloat16格式的机器学习工作负载设计的。AMD 还提供了半精度的功能,他们的软件栈显示了对bfloat16格式和IEEE 格式的支持[2]。AMD GPUs 半精度的理论性能分别遵循单/双精度的自然2/4加速。例如, Mi50 GPU 的理论FP16 性能为26.5 Tflop/s, 而FP32 为13.3 Tflop/s, FP64为6.6 Tflop/s。但也许最广泛使用的具有半精度功能的硬件是NVIDIA GPUs, 它自Pascal 架构以来就引入了半精度运算。在本节中,我们将重点介绍NVIDIA GPUs 及其数学库,以突出数字内核的半精度开发。

NVIDIA GPUs 实现了IEEE-754 标准定义的“binary16”格式[2]。虽然Pascal GPU 体系结构引入了对FP16 算子的硬件支持,但是为Summit 超级计算机提供动力的Volta 体系结构配备了用于FP16 中矩阵乘法的硬件加速单元(称为Tensor core)。这些Tensor core 理论上比前一种架构的理论FP16 峰值性能快12倍。利用Tensor core 的应用程序可以比在同一个GPU 上使用常规的FP16 算子快4 倍。Tensor core 还能够执行混合精度乘法,具有低精度输入(例如半精度)和更高精度输出(通常是单精度)。张量核LU在2.1.1节有更详细的讨论。

在半精度基本线性代数子例程(BLAS)方面,大多数可用的例程仅考虑稠密矩阵乘法(GEMMs)。从机器学习应用的角度来看,训练/推理中的大多数性能关键组件可以重新设计以利用GEMM 内核。至于稠密线性代数,许多高级算法被构建来从GEMM 调用中提取它们的高性能。因此,通过FP16 GEMM (HGEMM) 加速这些性能关键步骤,将使整个算法的性能优势得以传播,同时保持其他数值阶段的原始精度。这种实践的一个例子是混合精度稠密LU 分解[3], 它用于加速 $Ax = b$ 的双精度解,见第2.2节。

2.1.1 半精度硬件加速

CUDA 工具包是最早提供半精度(即FP16)运算的编程模型之一。2015年底,对基于Maxwell 架构的选定嵌入式GPU 模型增加了早期支持。自从Pascal 架构以来,FP16 算子已经成为支持CUDA 的GPUs 的主流。一般来说,半精度具有比单精度或双精度小得多的动态范围。

Volta 和 Turing 架构为 FP16 中的矩阵乘法引入了硬件加速。硬件加速单元叫做 Tensor core。它们可以提供比 FP32 峰值性能快 8 倍的理论峰值性能。例如，每个 Volta V100 GPU 有 640 个 Tensor core，均匀分布在 80 个多处理器上。每个 Tensor Core 拥有一个混合精度的 $4 \times 4 \times 4$ 矩阵处理数组来执行这个运算 $D = A \times B + C$ ，其中 A, B, C 和 D 是 4×4 矩阵。输入 A 和 B 必须用 FP16 格式表示，而 C 和 D 可以用 FP16 或 FP32 格式表示。 C 和 D 也可能指向同一个矩阵。

NVIDIA 的供应商库 cuBLAS 提供了各种优化的例程，主要是 GEMMs，如果进行了相应的配置，它们可以利用 Tensor Core 加速。例如，例程 cublasHgemm 实现了真正的 FP16 算子的 GEMM 操作。

除了供应商库之外，还可以通过使用编程模型提供的低级 API 来利用自定义内核中的 Tensor Core。如图 1 所示，Tensor Cores 通过称为片段的不透明数据结构处理输入和输出数据。每个片段用于存储一个矩阵。片段可以通过加载矩阵 `sync()` API 从共享内存或者全局内存中加载。还有一个类似的 API 可用于将输出片段的内容存储到 GPU 的共享/全局内存中。`mma_sync()` API 用于执行乘法运算。用户负责根据需要的声明片段，并以正确的顺序调用 APIs。

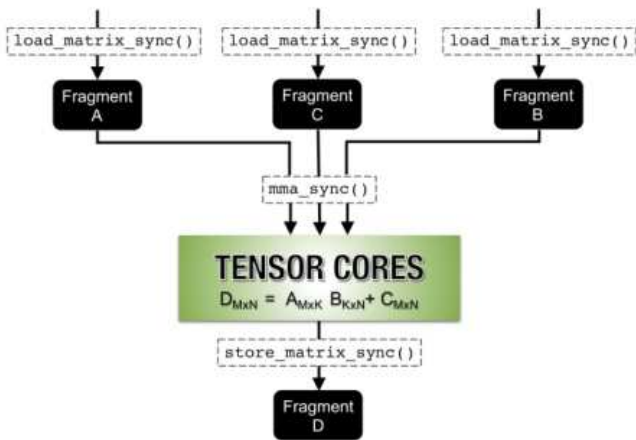


图 1 张量核心单元的可编程性

编程模型对张量核的编程施加了一些限制。首先，同样规模的控制片段 GEMM 维度 (M, N, K) 被限制为三个离散组合，即 $(16, 16, 16)$ 、 $(32, 8, 16)$ 和 $(8, 32, 16)$ 。其次，加载、存储和乘法操作必须由一个完整的经线（32 个线程）执行。最后，加载/存储 API 要求相应矩阵的前导维度是 16 字节的倍数。例如，规模为 $(16, 16, 16)$ 的标准 GEMM 操作需要三个加载矩阵 `load_matrix_sync()` 调用（对于 A 、 B 和 C ），一个 `mma_sync()` 调用，然后是最终存储矩阵 `store_matrix_sync()` 调用写入结果。迄今为止的最新 CUDA 版

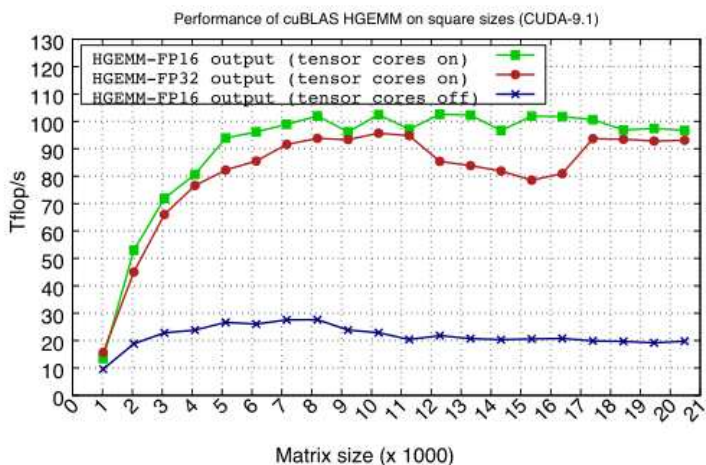


图 2 不同HGEMM内核和cuBLAS库在方形上的性能。结果用CUDA-9.1的特斯拉V100 GPU来展现

本(10.1) 通过名为mma.sync 的指令提供对张量核的直接访问。该指令允许一个warp 执行四个独立的规模为(8, 8, 4)的GEMM 操作。但是，随着新架构的发布，使用显式指令可能会导致开源库的长期兼容性问题。

2.1.2 半精度GEMM (HGEMM)

cuBLAS 库提供了几个利用可减少的FP16精确度的例程。图2 显示了三种不同的HGEMM 内核的性能。如果关闭张量内核，一个具有半精度输出的HGEMM 内核，可以达到30 Tflop/s 的性能。虽然这是单精度性能的2倍，但如果打开张量核，则可以实现显著更高的性能。如图所示，张量核能够提供100 Tflop/s 的渐近性能，这是非加速HGEMM渐近性能的5 倍。

然而，图2中最有趣的性能图可能是带有FP32输出的HGEMM。原因是它的性能接近于加速的HGEMM内核，但是在输出上有更高的精度。这对于混合精度算法来说尤为重要[3,4]。为了更全面地看待这个事实，图3显示了三个不同的HGEMM 内核之间的前向误差，相对于来自英特尔的单精度GEMM 内核。前后误差计算为：
$$\frac{\|R_{cuBLAS} - R_{MKL}\|_F}{\sqrt{k+2}|\alpha|\|A\|_F\|B\|_F+2|\beta|\|C\|_F}$$
其中 k 为矩阵规模。第一个令人惊讶的观察结果是，如果张量核打开，具有FP16 输出的HGEMM 操作会更准确，这意味着张量核心单元的利用可以实现更好的性能和更高的精确度。第二个观察结果是，与其他两个HGEMM变体相比，使用FP32 输出执行HGEMM?至少可以提高两位数的精确度。鉴于具有FP32 输出的HGEMM大多在峰值张量核心吞吐量的 90% 以内，对于旨在实现更高精度同时利用半精度优势的混合精度算法，它显然是最佳选择。

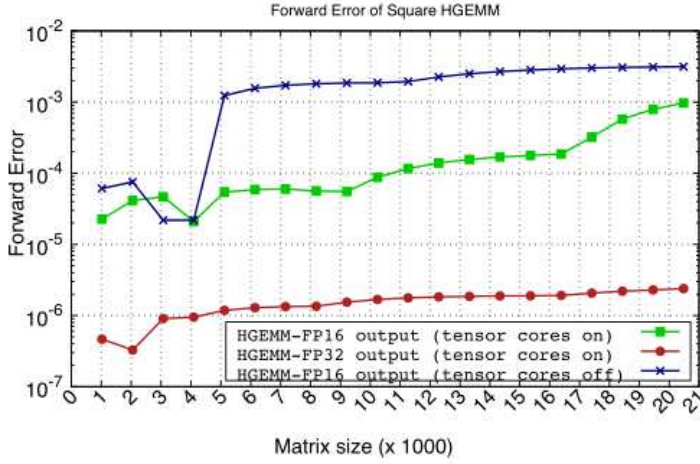


图 3 HGEMM相对于MKL SGEMM的向前误差($C = \alpha AB + \beta C$)。使用cuBLAS 9.1和MKL 2018.1显示了方形上的结果。向前误差为 $\frac{\|R_{cuBLAS} - R_{MKL}\|_F}{\sqrt{k+2}|\alpha|\|A\|_F\|B\|_F + 2|\beta|\|C\|_F}$ ，其中 k 为矩阵的阶数

2.1.3 批量(HGEMM)

除了供应商提供的BLAS，几乎没有人致力于构建利用NVIDIA GPU的张量核的开源BLAS例程。在这方面的一个例子是MAGMA库[5]，它有一批利用张量核的HGEMM内核[6]。内核在张量核上建立一个抽象层来克服它们的规模限制，这样内核就可以使用任意的规模的矩阵。MAGMA中的批量HGEMM内核在相对较小的规模上优于cuBLAS，如图4所示。同样的工作也表明，规模极小的矩阵(例如其规模为10的矩阵)可能不一定受益于张量核的加速。

2.2 经典迭代细化

在现代架构中，32位操作的性能通常至少是64位操作的两倍。这有两个原因。首先，在大多数现代处理器上，32位浮点运算的速度通常是64位浮点运算的两倍。其次，通过内存系统移动的字节数减少了一半。

混合精度算法来源于观察，在许多情况下，一个问题的单精度解可以细化到达到双精度的精确度。例如，细化可以通过牛顿算法(见方程(1))来完成，该算法根据迭代公式计算函数 $f(x)$ 的零点。

$$\frac{\|R_{cuBLAS} - R_{MKL}\|_F}{\sqrt{k+2}|\alpha|\|A\|_F\|B\|_F + 2|\beta|\|C\|_F}$$

一般情况下，我们会在单精度运算中计算一个起始点和 $f(x)$ ，而细化过程将在双精度运算中计算。

如果细化过程比解的初始计算成本更低，则双精度精确度可以达到与单精度精确

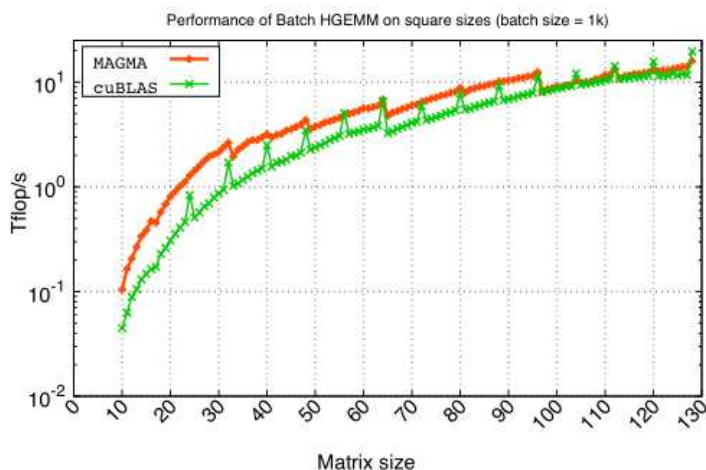


图 4 批量HGEMM内核在方形上的性能。结果用CUDA-9.1的特斯拉V100 GPU 显示

度几乎相同的速度。

求解稠密或稀疏线性系统的一种常用方法是使用高斯消元法对系数矩阵进行LU 分解。首先，将系数矩阵A 分解为一个下三角矩阵L 和一个上三角矩阵U 的乘积。该系统的解是通过首先求解 $Ly = Pb$ (前向替换)，然后求解 $Ux = y$ (后向替换)来实现的。由于舍入误差，计算出的解X 带有一个由系数矩阵A 的条件数放大的数值误差。

为了改进计算结果，我们可以应用一个迭代过程，该过程在每次迭代时对计算结果进行校正，然后得到通常称为迭代细化(IR)算法的方法。正如Demmel 指出的[7]，舍入误差的非线性使得迭代细化过程等效于应用于函数 $f(x) = b - Ax$ 的Newton 方法。只要系统不是太病态，算法产生一个精确到有效精度的解。双精度/双精度的迭代细化是一个相当好理解的概念，Wilkinson [8]，Moler [9]和Stewart [10]对其进行了分析。

迭代细化是一个长期存在的方法，是由威尔金森在1940年代编程的，用于ACE 数字计算机。其思想是通过迭代求解一个校正方程并在当前解中加入校正值来改进线性系统的计算解。[11, Chap. 12]。

该算法可以修改为使用混合精度的方法。三个任务的原始求解/因式分解、残差计算和校正方程的求解可以在相同的精度(固定精度)或不同的精度(混合精度)下完成。最初的使用是混合精度，以有效精度的两倍计算残差。

本文采用单精度运算计算了 $PA = LU$ 的分解，三角形系统 $Ly = pb$ 和 $Ux = y$ 的解。残差计算和解的更新使用双精度运算和原始双精度系数计算(见算法1)。计算成本最高的运算，系数矩阵A 的因式分解，使用单精度运算，利用其更高的速度。唯一必须以双精度执行的运算是残差计算和解的更新(在算法1中用 ε_d 表示)。

算法1: 混合精度, 直接求解器的迭代优化

```

1:  $LU \leftarrow PA$   $\triangleright (\epsilon_s)$ 
2: 求解  $Ly = Pb$   $\triangleright (\epsilon_s)$ 
3: 求解  $Ux_0 = y$   $\triangleright (\epsilon_s)$ 
4: for  $k = 1, 2, \dots$  do
5:    $r_k \leftarrow b - Ax_{k-1}$   $\triangleright (\epsilon_d)$ 
6:   求解  $Ly = Pr_k$   $\triangleright (\epsilon_s)$ 
7:   求解  $Uz_k = y$   $\triangleright (\epsilon_s)$ 
8:    $x_k \leftarrow x_{k-1} + z_k$   $\triangleright (\epsilon_d)$ 
9:   检查收敛性
10: end for

```

我们观察到, 只有计算复杂度为 $o(n^3)$ 的运算是利用单精度处理的, 而以双精度执行的所有运算最多为 $o(n^2)$ 复杂度。对于LU 分解, 系数矩阵A转换为单精度, 得到的因子以单精度存储, 而初始系数矩阵A 需要保存在内存中。因此, 下面这种方法的一个缺点是, 它使用的内存比标准双精度运算多使用 50%。算法1中的方法在许多情况下可以对稀疏线性系统的解提供显著的改进, 如果:

- 1.单精度计算显著快于双精度计算;
- 2.迭代细化过程在少量步骤中收敛;

3.与系统分解的成本相比, 每次迭代的成本是很小的。如果每次迭代的成本太高, 那么较少的迭代次数将导致相对于全双精度求解器的性能损失。在稀疏情况下, 对于一个固定规模的矩阵, 系统分解的成本和迭代细化步骤的成本可能会根据非零的数量和矩阵稀疏结构的不同而有很大的差异; 这将在第4.2节中讨论。在稠密的情况下, 结果更可预测。

请注意, 在迭代细化过程中停止判据的选择是至关重要的。例如, 可以获得算法1的每一步计算的误差[12,13]。最近, Carson 和highham [14]分析了一种三精度迭代细化方案(分解精度、有效精度、残差精度), 得出如果矩阵A的条件数不太大, 即 $\kappa_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty < 10^4$

然后将FP16 用于 $o(n^3)$ 部分(LU) 分解和(FP32,FP64) 或(FP64,FP128)作为(有效, 残差)精度用于 $o(n^2)$ 部分(细化循环), 可以得到分别为 $10^{(-8)}$ 和 $10^{(-16)}$ 数量级的前向误差和后向误差。我们注意到, 如果X 是方程 $Ax=b$ 的一个近似解。前向误差定义为: $\frac{\|\hat{x}-x\|_\infty}{\|x\|_\infty}$ 后向误差定义为: $\min\{\epsilon : (A + \Delta A)\hat{x} = b, \|\Delta A\| \leq \epsilon\|A\|\}$ 可以评估

为: $\frac{\|r\|_2}{\|A\|_2\|\hat{x}\|_2}$, where $r = b - A\hat{x}$.

2.3 GMRES-IR

Carson 和highham [15]提出了用FP16 LU 因子预处理的广义最小残差(GMRES)[16]作为校正方程的求解器,并证明在这种情况下,当(有效、残差)精度为(FP32、FP64)时,条件数的约束可以放宽到 $\mathcal{K}_\infty(A) < 10^8$ 当(有效、残差)精度为(FP64、FP128)时,条件数的约束可以达到 10^{12} 。限制条件数和前向,反向误差参考[17, 表2.2]。在残差精度等效于有效精度时,[17]给出了这种基于GRMES 的方法的分析。。

其思想是,GMRES 求解器将为 $Az_k = r_k$ 提供比基本三角形求解器更好、更稳定的近似解,而基本三角形求解器直接受低精度LU 因子的质量影响。利用GMRES 方法,我们仍然可以保证校正方程 $Az_k = r_k$ 的解有一些正确的数位和在算法要求的收敛判据上的残差。细化过程的收敛判据被选择为分解过程中使用的低精度运算的最大相对误差的阶数(例如,当LU 分解分别在FP16或FP32中时,我们使用 10^{-4} 或 10^{-8})。这种变体被Carson 和Highham 称为基于GMRES 的迭代细化(GMRES-IR) ,并在算法2中描述。注意, U^{-1} 和 L^{-1} 从来没有明确形成;取而代之的是,GMRES所需的矩阵向量乘积 $U^{-1}L^{-1}Ay$ 是通过A和两个三角形解的乘积来计算的。由于本文的重点是实际使用和可能的性能增益,而不是误差分析,因此我们建议读者参考[11,15,17],以获得关于IR和GMRES-IR技术的详细误差分析。 U^{-1} 和 L^{-1} 从未明确形成;相反,GMRES 所需的向量乘积 $U^{-1}L^{-1}Ay$ 是通过乘以A和两个三角求解来计算的。由于本文侧重于实际使用和可能的性能增益而不是错误分析,我们将读者指向[11,15,17]以了解IR和GMRES-IR技术的具体误差分析。

算法2: 基于GMRES (GMRES- IR)的迭代优化

- 1: 将A转化为 A_f , 从精度 u_w 到 u_f
 - 2: 在精度 u_f 中对 A_f 进行LU分解
 - 3: 利用精度 u_f 计算得到的 A_f 的LU分解求得初始解 x_0 , 然后将 x_0 转化为精度 u_w
 - 4: 细化循环, 外层循环
 - 5: **repeat**
 - 6: 在精度 u_r 下计算残差 $r_k = b - Ax_k$, 并将其转换为 u_w ▷ 残差
 - 7: 在精度 u_w 下通过GMRES求解 $U^{-1}L^{-1}Az_k = U^{-1}L^{-1}r_k$ ▷ 修正
 - 8: 在精度 u_w 下修正当前解 $x_{k+1} = x_k + z_k$ ▷ 更新
 - 9: **until** x_k 足够精确
-

设计和实现有效利用当前高度并行计算机体系结构的数值算法是一个挑战,特别

是当要实现接近峰值性能时。由于在GMRES-IR方法中, $O(n^3)$ 运算量的精度较低, 这种思想允许一类新的迭代优化求解器和一些计算技术, 使我们能够解决基本的问题 $Ax = b$, 接近FP64 的峰值性能。这些发展为今后的工作开辟了新的方向, 包括进一步优化、开发一套混合精度分解、线性系统求解器、特征求解器和奇异值分解(SVD)。

2.3.1 放缩

很明显, 在迭代细化中使用低精度浮点运算可以显著提高速度。但是, FP16的动态范围很小, 因此很可能会遇到上溢(值超过了该类型所能表示的最大值)、下溢(值低于该类型所能表示的最小值)和次规格化数字(指数部分非常小的数字, 这些数字的有效数部分首位用0 来表示)。为了解决这些问题, 现在我们讨论缩放算法, 这些算法在[18]、[19]和[20]中提出。我们向感兴趣的读者推荐这些参考资料以获取更多信息。

我们在转换为FP16之前考虑了双边对角缩放: A 被 RAS 替换, 其中

$$R = \text{diag}(r_i), \quad S = \text{diag}(s_i), \quad r_i, s_i > 0, \quad i = 1:n.$$

这种缩放算法是在线性系统和线性规划问题的背景下发展起来的。尽管有大量关于如何缩放这些问题的文献, 但对于何时或如何缩放这些问题还没有明确的结论; 最近的一项实验研究见[21]。与以往的研究相比, 我们缩放的目的是减少一个条件数或加快一个迭代法应用于缩放矩阵的收敛速度, 我们缩放的目的是帮助将一个单精度或双精度矩阵压缩为半精度矩阵, 特别是应用由此产生的半精度LU 因子进行迭代细化。我们在算法3中给出了双边对角缩放的用法。

算法3: (两边对角缩放然后舍入)。该算法将 $A \in \mathbb{R}^{n \times n}$ 舍入到fp16矩阵 $A^{(h)}$, 缩放所有元素以避免溢出。 θ 是 $(0, 1]$ 上的一个参数。

- 1: 对 A 应用任意两侧对角线放缩算法, 得到对角矩阵 R 和 S 。
 - 2: 令 β 是 RAS 中元素的最大值
 - 3: $\mu = \theta x_{\max} / \beta$
 - 4: $A^{(h)} = fl_h(\mu(RAS))$
-

我们考虑两种不同的算法来确定 R 和 S ; 两种算法都在有效精度下执行。一种选择是行列平衡 (A 被 $\text{diag}(R) \cdot A \cdot \text{diag}(C)$ 所取代), 它确保每一行和每一列的最大元素的模等于1- 也就是说, 每一行和每一列都是平衡的。LAPACK 例程xyyEQU 执行这种缩放形式[22]。Knight、Ruiz、和Ucar [23]提出了一种保对称的两边缩放。该算法是迭代的, 并且在两边同时缩放, 而不是按顺序在一边然后在另一边依次缩放。

2.4 混合精度分解

Haidar et al. [3]提出使用混合精度因子分解的IR 方法。虽然经典的IR 和GMRES-IR 等扩展使用固定精度分解(例如, 在算法2所示的精度 u_f), 但混合精度分解在算法的关键部分应用更高的精度(例如 u_w), 以获得超精密分解, 同时保留低精度对应部分的性能。这些发展应用于GPU 张量核, 说明FP16 可以用来获得FP64 精确度的问题, 与更典型的要求 $k_\infty(A) < 10^4$ 相比, $k_\infty(A)$ 高达 10^5 。这项工作说明混合精度技术对于许多工程领域的线性求解器来说是非常有意义的。结果表明, 在单个NVIDIA V100 GPU 上, 新的求解器可以比优化的双精度求解器快4 倍[3], [4], [24]。

混合精度分解的动机是在使用非常低的精度时需要获得额外的精度, 比如FP16。而且, 这可以很容易地克服实现问题以及使用FP16算子的其他限制, 从而利用专用硬件的能力, 如GPU张量核, 用于更大范围的科学计算应用程序。

混合精度分解的构建模块是混合精度BLAS。拥有混合精度BLAS, 可以轻松开发多种混合精度LAPACK算法。目前, cuBLAS提供混合FP32-FP16 精度HGEMM, 使用GPU的张量核FP16加速。在这个GEMM中, 输入矩阵A和矩阵B可以是FP32, 在内部被转换到FP16, 用于在完全(FP32)精确度的张量核上计算一个GEMM, 结果存储在FP32的GPU内存中。使用这种混合精度BLAS例程有两个主要好处。首先, 请注意这种混合精度的HGEMM几乎与非混合FP16 精度的HGEMM一样快(参见图2), 其次, 使用混合精度增益约一个小数点后的精确度(参见图3)。

除了上面列出的两个主要优点之外, 混合精度GEMM的可用性也使我们能够轻松开发其他混合精度算法, 例如LAPACK, 特别是我们最近在MAGMA[3]中添加的各种混合精度分解。图5显示了混合精度LU(标记为“FP16-TC hgetrf LU”)的性能。注意到, 这种分解比dgetrf快4-5倍。它的数据存储在FP32 中, 实现与sgetrf 相同, 只是它使用混合精度HGEMMs来更新后续矩阵。

图6显示了MAGMA[3]中混合精度的迭代细化。4倍整体加速器是由于一些优化。首先, 与hgetrf 性能(24 Tflop/s vs. 26 Tflop/s)相比, 达到FP64精度的3次迭代导致了约2 Tflop/s的损失, 即一次迭代的成本可以推断为约2%。例如, 通过多达40次迭代丢失75%, 将导致与FP64求解器相比没有加速。这种每次迭代的成本非常低, 这是因为所有的数据转换都与计算内核融合在一起。如果没有融合, 负载很容易就会高出3倍。其次, 注意使用混合精度分解的迭代细化在求解的迭代方面有2倍以上的成本(3次迭代vs. 7 次迭代直到FP64收敛)。这是因为混合精度HGEMM比FP16 HGEMM具有精确的额外数位, 即转化为更精确的混合精度LU。

2.5 Cholesky分解

在上一节中, 我们考虑了一般对称矩阵的缩放, 我们现在假设在精度为 u 的有限

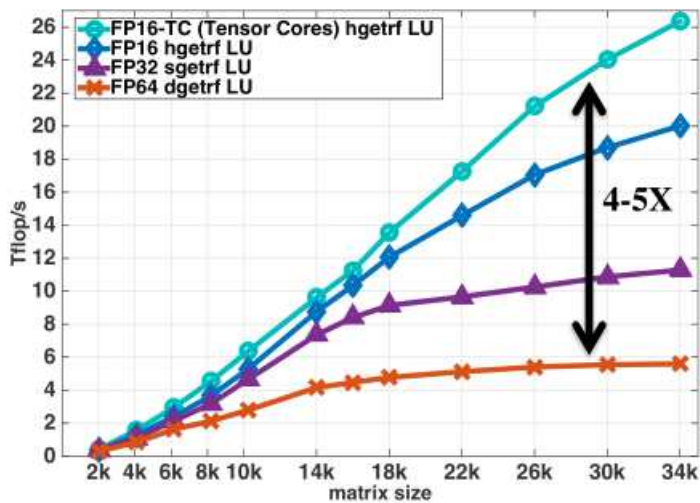


图 5 MAGMA中的混合精度LU(hgetrf)及其与FP64LU的加速比

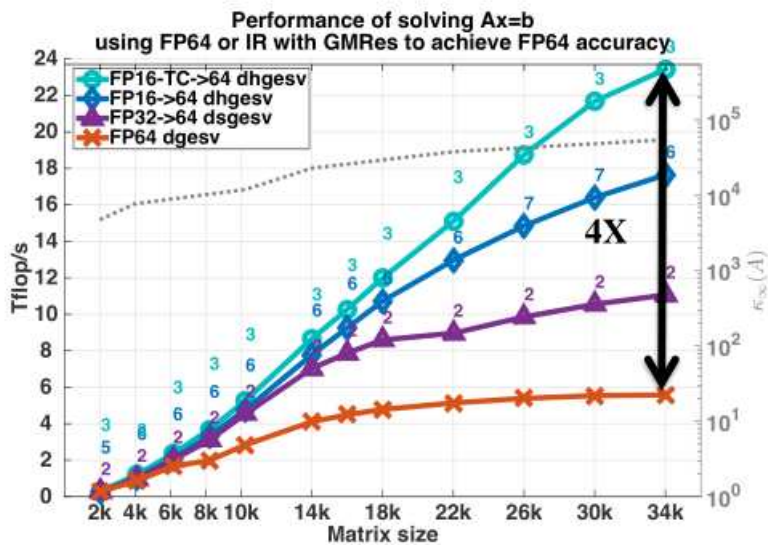


图 6 混合精度迭代细化的MAGMA和加速与FP64求解器。注:每次迭代的开销 $\approx 2\%$, 与常规FP16 LU相比, 混合精度LU的迭代开销减少了2倍以上(在FP64收敛之前, 迭代次数为3次, 迭代次数为7次)

精度运算中, 给定对称正定矩阵 $A \in \mathbb{R}^{n \times n}$, 并希望在精度为 $u_h > u$ 的有限精度运算中计算Cholesky 分解。最重要的实际案例是 $(u_h, u) = (\text{half}, \text{single}), (\text{half}, \text{double}),$ 或 $(\text{single}, \text{double})$ 。自然地, $A^{(h)} = fl_h(A)$, 其中 fl_h 表示精确到 u_h 舍入运算然后精确地计算 $A(h)$ 的Cholesky 分解。对于半精度来说, 这个过程可能会失败, 原因有二。首先, 如果使用fp16, 那么有限的范围可能会导致在舍入阶段上溢。其次, 对于bfloat16和fp16, $A(h)$ 可能不是(充分)正定的, 因为对于单精度或双精度而言, 其最小特征值安全地远离零的矩阵可能在舍入到半精度引起的扰动下在数值上变得不确定。为了克服这些问题, 我们将使用缩放和移位。

2.5.1 缩放

第一步是将矩阵 H 缩放到 $H = D^{-1}AD^{-1}$, $D = \text{diag}(a_{ii}^{1/2})$, D 将保持精度 u 。由于Cholesky 分解在双边对角缩放下基本上是数值不变的(从Cholesky 因子的递推关系可以看出), 因此, 对于fp16, 缩放的唯一原因是为了减少动态范围以避免上溢并减少下溢的机会。对于bfloat16 或单精度, 通常不必缩放, 我们可以使用 A 贯穿。在接下来的演示中, 为了简化, 我们将一直缩放。在LU 分解的背景下[18] 使用了舍入到低精度之前的双边对角缩放。那里使用缩放平衡行和列; 我们使用

的缩放, D 是对称正定矩阵的自然类比。对于Cholesky 分解, 我们需要一个额外的部分来确保分解的成功, 我们接下来考虑这个问题。

2.5.2 移位

我们现在将 H 转换为较低精度的 u_h , 并入一个移位以确保较低精度的矩阵足够正定, 可以成功进行Cholesky分解, 如[19, Sec.2]所述。为获得 $G = H + c_n u_h I$ 我们将通过 $c_n u_h I$ 移位 H , 其中 c_n 是一个正整数。由于 H 的对角部分是 I , 无论是以精度 u 移位然后舍入或舍入然后在以精度 u_h 移位, 移位都不会产生舍入误差。

对于FP16, 由于动态范围较窄, 我们还将移位矩阵乘以一个标量, 使其接近上溢水平 $x_{\max}$, 以尽量减少正在产生下溢和次规范输的机会。所以我们最终地构造精度 u_h 矩阵的格式为

$$G = H + c_n u_h I,$$

$$\beta = 1 + c_n u_h, \quad \mu = \theta x_{\max} / \beta,$$

$$A^{(h)} = fl_h(\mu G),$$

其中 $\theta \in (0, 1)$ 是一个参数。注意 $\beta = \max_{ij} |g_{ij}|$, 所以 $A(h)$ 中的任意元素的最大绝对值是 $\beta = \max_{ij} |g_{ij}|$ 。还要注意, 由于Cholesky 分解的增长因子是1 (see, e.g., [11, Prob. 10.4]), 在 $A(h)$ 的Cholesky 因式分解过程中没有上溢的危险。

我们参考[19, Sec. 3.3]对 c_n 选择进行了分析。然而, 由于估计是保守的, 我们将 c_n 取

为小的常数 c 。如果Cholesky 分解失败，我们将增加 c 并重试。我们将通过实验确定对于一系列感兴趣的问题， c 应该取多大。在此基础上，我们给出算法4 中的低精度Cholesky 分解算法。

2.6 最小二乘问题的迭代细化

我们考虑线性最小二乘问题 $\min_x \|Ax - b\|_2$ ，其中 $A \in \mathbb{R}^{m \times n}$ ，且 $m \geq n$ 具有满秩。混合精度迭代细化和GMRES-IR的思想可以适用于最小二乘情况。最小二乘问题在实践中可能条件是病态的，因此舍入误差可能会导致解不够精确。在这种情况下，可以使用迭代求精来提高精度，同时也可以提高稳定性。

算法4: (精度为 u_h 的Cholesky因式分解)给定一个精度为 u 的对称正定 $A \in \mathbb{R}^{n \times n}$ ，该算法计算精度为 u_h 的近似Cholesky因式分解 $\mu D^{-1} A D^{-1}$ ，其中 $D = \text{diag}(a_{ii}^{1/2})$ 。标量 $\theta \in (0, 1]$ 和正整数 c 是参数。

```

1:  $D = \text{diag}(a_{ii}^{1/2})$ ,  $H = D^{-1} A D^{-1}$   %设 $h_{ii} \equiv 1$ 而不是计算它。
2:  $G = H + cu_h I$ 
3:  $\beta = 1 + cu_h$ 
4:  $\mu = \theta x_{\max} / \beta$ 
5:  $A^{(h)} = fl_h(\mu G)$ 
6: 尝试用精度为 $u_h$ 的Cholesky因式分解 $A^{(h)} = R^T R$ 。
7: if Cholesky因式分解失败, then
8:    $c \leftarrow 2c$ , 转到行2
9: end if
```

2.6.1 基于Cholesky的方法

正规方程思想使用 $A^T A$ 的Cholesky因式分解求解了

$$A^T A x = A^T b$$

(见2.5节)。一般而言，数值分析家不赞成使用这种方法，因为它有一个 $\kappa_2(A)u$ 阶的后向误差约束[11, 第20.4节]并且Cholesky 因式分解对于 $\kappa_2(A) > u^{-1/2}$ 是不成立的，但统计学家使用它是有一定道理的[25]。这里，我们假设 A 是(充分)条件良好的。我们提出了算法5中给出的基于GMRES-IR的最小二乘求解法。

算法5: (基于Cholesky的GMRES-IR算法求解最小二乘问题) 设一个满秩的 $A \in \mathbb{R}^{m \times n}$, 其中 $m \geq n$, $b \in \mathbb{R}^m$ 以精度 u 给出。该算法利用基于Cholesky 的GMRES-IR算法解决了最小二乘问题 $\min_x \|b - Ax\|_2$ 。标量 $\theta \in (0, 1]$ 和正整数 c 是参数。

```

1:  $B = AS$ , 其中  $S = \text{diag}(1/\|a_j\|_2)$ ,  $a_j$  表示  $A$  的第  $j$  列.
2:  $\mu = \theta x_{\max}$ 
3:  $B^{(h)} = fl_h(\mu^{1/2} B)$ 
4: 以精确度  $u_h$  计算出  $C = B^{(h)T} B^{(h)}$ 
5: 以精确度  $u_h$  计算出  $Cholesky$  因式分解  $C + cu_h \text{diag}(c_{ii}) = R^T R$ .
6: if  $Cholesky$  因式分解失败, then
7:    $c \leftarrow 2c$ , 转到行5
8: end if
9: 形成  $b^{(h)} = fl_h(SA^T b)$ 
10: 以精确度  $u_h$  计算出  $R^T R y_0 = b^{(h)}$  且在精确度  $u$  下形成  $x_0 = \mu S y_0$ 
11: for  $i = 0 : i_{\max} - 1$  do
12:   以精确度  $u_r$  计算出  $r_i = A^T(b - Ax_i)$  并且以精确度  $u$  将  $r_i$  四舍五入。
13:   用GMRES以精确度  $u$  求解  $MA^T A d_i = M r_i$ , 其中  $M = \mu S R^{-1} R^{-T} S$  并且矩阵向
   量内积  $A^T A$  以精度  $u_r$  计算, 并以精确度  $u$  存储  $d_i$ 
14:   在精确度  $u$  下  $x_{i+1} = x_i + d_i$ 。
15:   if 收敛, then
16:     返回  $x_{i+1}$ , quit
17:   end if
18: end for

```

我们对该算法给出了一些评论。第1行生成具有单位2-范数的列的矩阵 B 。第4行的计算 $C = B^{(h)T} B^{(h)}$ 生成了一个对角元素为常数 $\mu = \theta x_{\max}$ 的对称正定矩阵, 故 $\theta < 1$ 时不会发生溢出。第5行上的移位与算法4 中类似, 但这里矩阵 C 已经有了很好比例和精度为 u_h , 所以不需要缩放 C 以获得单位对角线。

从数值稳定性的角度来看, 算法5中明确形成 $C = B^{(h)T} B^{(h)}$ 是有两个原因。首先 C 是用来形成一个预处理程序, 所以形成叉乘矩阵的通常问题(丧失重要性和条件平方)就不值得关注了。其次, 如果我們是在NVIDIA V100上的fp16中操作, 我们可以利用张量核在形成 C 时积累单精度的块融合乘法运算的; 这产生了一个更精确的 C , 如Blanchard 等人的误差分析所示[26]。

为计算 $\hat{R}$, 我们有

$$\hat{R}^T \hat{R} \approx B^{(h)T} B^{(h)} \approx \mu S A^T A S,$$

或者

$$(A^T A)^{-1} \approx \mu S \hat{R}^{-1} \hat{R}^{-T} S,$$

所以我们用近似于 $A^T A$ 逆的方法进行预处理。我们对精度为 u_r 的向量应用预处理算子 $M A^T A$ 。计算 $y = A^T A x$ 需要花费 $4mn$ 次运算, 计算 $S R^{-1} R^{-T} y$ 需要花费另外 $2n^2 + n$ 次运算, 总共需要 $4mn + 2n^2 + n$ 次运算。对于 $m \gg n$ 和较大的 n , 计算 $y = A^T A x$ 比形成 $A^T A$ 所需的 mn^2 次运算少 $n/4$ 倍, 而对于 $m \approx n$, 差别是 $n/6$ 倍。对于较大的 n , 即使考虑到我们比较的是不同的精度的运算速度, 只要GMRES快速收敛, 与形成 $A^T A$ 和计算Cholesky 因式分解的成本相比, 细化阶段的成本应该是可以忽略的。

与此工作相关的是用于计算QR 分解 $A=QR$ 的Cholesky-QR算法。它形成叉积矩阵 $A^T A$, 计算Cholesky 因子分解 $A^T A = R^T R$, 则得到正交因子 Q 为 $Q = A R^{-1}$, 该过程可迭代以获得更好的数值稳定性; 例如, [27], [28], [29], [30]。我们工作的不同之处在于我们不计算 Q , 我们在低于工作精度的情况下进行Cholesky 因子分解, 我们用预条件的迭代细化法解决最小二乘问题。

2.6.2 增广矩阵方法

Carson、Higham和Pranesh在[20]中提出了另一种混合精度最小二乘迭代精炼的方法, 该方法是基于使用QR因式分解的方法

$$A = Q \begin{bmatrix} R \\ 0 \end{bmatrix}$$

其中 $Q = [Q_1, Q_2] \in \mathbb{R}^{m \times m}$ 是 $Q_1 \in \mathbb{R}^{m \times n}$ 和 $Q_2 \in \mathbb{R}^{m \times (m-n)}$ 的正交矩阵, 并且 $R \in \mathbb{R}^{n \times n}$ 是上三角形。唯一的最小二乘解是 $x = R^{-1} Q_1^T b$, 残差是 $\|b - Ax\|_2 = \|Q_2^T b\|_2$

Björck [31] 提出了一种迭代细化的方法, 即使在 $Ax = b$ 不一致的情况下也能奏效, 细化是在增广系统上进行的

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}$$

它等价于正规方程。通过这种方法, 可以同时细化最小二乘问题的解 x_i 和残差 r_i , Björck [31] 表明利用 A 的QR 因子可以求解线性方程组。

现有的关于有限精度下该方法的收敛性和准确性的分析假设最多使用两个精度; 工作精度 u 用于计算QR因子分解, 求解增广系统, 并计算更新。用第二精度 $u_r \leq u$ 来计算

残差。通常 $u_r = u^2$ ，这种情况下可以证明，只要增广系统矩阵的条件数小于 u^{-1} ，细化过程将以 u 数量级的极限前向误差收敛;参见[32]和[11, 20.5节]及其参考文献。

结果表明，Carson和Higham[14] 的三精度迭代求精方法可以适用于这种情况;对于线性[14]中提出的关于线性系统迭代细化的前向误差以及正态和分量的向后误差定理也适用。唯一必须改变的是对求解修正方程的方法的分析，因为我们现在使用的是A的QR因式分解，它可以以各种方式使用。

[20]的工作还将[15]中基于GMRES 的细化方案扩展到最小二乘情况，并表明可以利用A 的现有QR 因子构造左预处理器，使得GMRES可证明收敛到预处理增广系统的后向稳定解。此外，还表明现有的鞍点系统开发的预处理器在实践中也能很好地用于基于GMRES 的方法，尽管误差分析并不适用。我们请读者参阅[20]了解更多细节。

2.7 量化整数LU分解

量化是一种被广泛应用于深度学习推理的技术[33,34]。虽然模型通常仍然以单精度训练，但量化压缩数据并使用较低的精度在推理阶段进行计算，即将训练后的模型应用到新的数据中，以便实际应用。对于 *int8* 量化模型，数据被转换成8位整数。与32位单精度相比，计算量和通信量减少了4倍，而精度损失是可以接受的(预测模型通常小于1%)。整数运算可用于大多数硬件架构。现场可编程门阵列(FPGA)通常更擅长整数运算，并且可能没有浮点数算术单元。用于深度学习推理的新型专用集成电路(ASIC)也正朝着将整数运算用于量化神经网络的方向发展。这促使我们研究使用整数运算进行部分旋转的高斯消元(LU分解)。

2.7.1 量化整数LU算法

存储格式	i 是32位整数
实数表示	$R(i) = i/2^{32} \times 2^0$
从双精度数 α 转换	$i \leftarrow \text{int32} (\alpha \times 2^{32})$
转化为双精度数 α	$\alpha \leftarrow \text{double} (i)/2^{32}$
加法	$R(i) + R(j) = i/2^{32} + j/2^{32} = (i + j)/2^{32} = R(i + j)$
乘法	$R(i) \times R(j) = i/2^{32} \times j/2^{32} = (i \times j)/2^{64} = (i \times j/2^{32}) / 2^{32} = R(i \times j/2^{32})$

表 1 给定的定点数的表示

基本思想是缩小数字的规模以适应定点数的表示: $i/2^{32} \times 2^0$ ，其中 i 是一个32位整数。由于指数在加法或乘法时不会改变，因此可以忽略。下面的加法形式是简单的整

算法6: 基于整数运算的部分旋转的LU分解

```
1: Input:  $n$  by  $n$  matrix  $A$  in double precision. Integer  $r$  for the range while normalizing  $A$ .
2: Declare identity matrix  $P$  as permutation matrix.
3:  $m \leftarrow \max(A) \times 2^r; A \leftarrow A/m$   $\triangleright$  将A标准化为 $[-2^{-r}, 2^{-r}]$ 
4:  $A_{int} \leftarrow \text{int32}(A \times 2^{32})$   $\triangleright$  将A转化为定点数表示
5: for  $i = 1, \dots, n$ , do  $\triangleright$  列上的主循环
6:    $\text{pivot} \leftarrow (\arg \max |A_{int}[i : n, i]|) + i - 1$   $\triangleright$  找到枢轴索引
7:    $\text{swap}(A_{int}[i, :], A_{int}[\text{pivot}, :])$   $\triangleright$  交换行
8:    $\text{swap}(P[i, :], P[\text{pivot}, :])$ 
9:    $\alpha \leftarrow \text{int } 64 (2^{32}) / A[i, i]$   $\triangleright$  使用整数除法
10:   $A_{int}[i : n, i] \leftarrow \alpha A_{int}[i : n, i]$   $\triangleright$  放缩列
11:   $A_{int}[i + 1 : n, i + 1 : n] \leftarrow A_{int}[i + 1 : n, i + 1 : n] - A_{int}[i + 1 : n, i] \times A_{int}[i, i + 1 : n] / 2^{32}$ 
12:   $\triangleright$ 使用整数移位的除法更新Integer rank-1
13: end for
14:  $L \leftarrow$  lower triangular part of double  $(A)/2^{32}$  with unit diagonal.
15:  $U \leftarrow$  upper triangular part of double  $(A)/2^{32}$  with unit diagonal.
16: Return:  $P, L, U$  as the result of factorization such that  $P(A/m) = LU$ 
```

数加法。乘法变成: $i/2^{32} \times j/2^{32} = (i \times j)/2^{64} = (i \times j/2^{32})/2^{32}$. 计算 $(i \times j/2^{32})$ 可以通过32位整数间的乘法, 然后返回64位整数的前边的32位。注意到这个操作可以用包含 *x64* 和 *ARM* 的现代CPU指令集架构的一条指令来完成。表1总结了提出的固定点数字表示法。表1总结了给定的定点数的表示。

算法6显示了基于整数运算的部分旋转的LU分解。循环中的计算主要是32位整数运算。第9行需要64位整数除法, 但每列只需要一次。第10行的刻度将保持在int32范围内, 因为枢轴的大小比列中的其他元素大。第11行的更新是32位整数乘法, 但是我们只需要64位结果中的前32位。

输入整数 r 决定将我们在将 A 转换为整数时实际使用了 $(32 - r)$ 位数。因为矩阵在分解过程中会增长, 并且我们在分解过程中没有任何动态缩放, 所以它可能会在某个时候达到整数范围并溢出。为了避免这种情况, 我们首先将矩阵缩放到 $[-2^{-r}, 2^{-r}]$ 。 r 越大, 我们在整数范围内的范围就越大。但是输入矩阵在转换成int32之后就不那么精确了。

2.7.2 量化整数LU数值结果

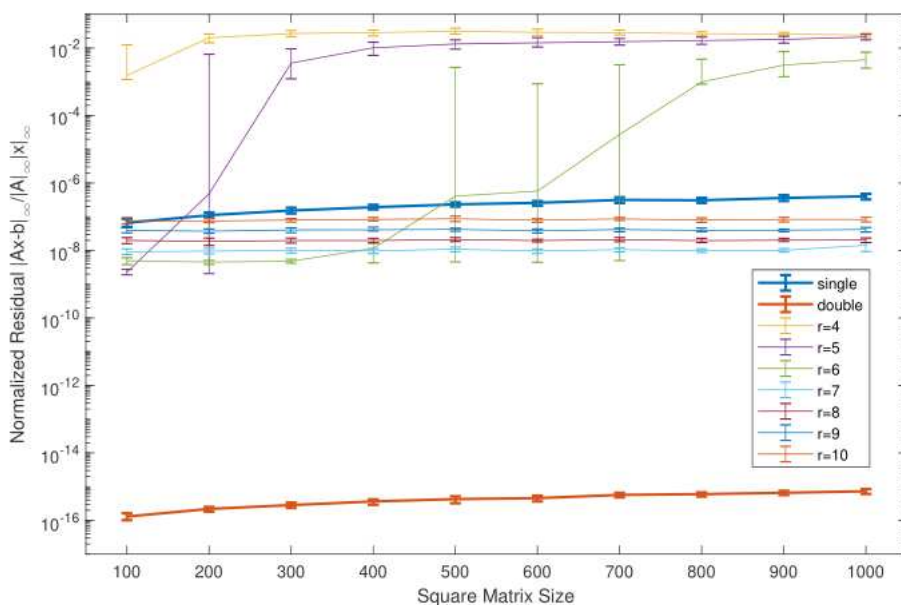


图 7 不同可用位数 r 的归一化后向误差与输入矩阵大小的关系。还显示了单精度和双精度的结果

图7展示了相对于输入矩阵的标准化的向后误差 $\|Ax - b\|_\infty / \|A\|_\infty \|x\|_\infty$ 。该算法是在MATLAB R2018b 中实现的。矩阵的每个元素都是 $(-1,1)$ 的均匀分布。每个点是30 个随机矩阵的几何平均值，误差线表示15% 和85% 的百分比。作为参考，单精度和双精度的LU分解的结果也以粗体线表示。对于大于 10^{-6} 的结果，发生溢出，算法失败。否则在分解过程中就没有数值误差了。该错误仅在浮点和定点格式之间的转换中引入，而不在整数分解过程中引入。向后错误随着 r 的增加而增加，因为在转换时输入被截断得更多。但当 $r = 10$ 时，它仍然使用 $32 - 10 = 22$ 位，结果可以与使用23尾数位的单精度相比。

2.7.3 量化整数LU的未来展望

我们想展示使用整数运算进行低精度分解的情况。下一步我们将进行更详细的误差分析。扩展为更短的整数，如int16和int8。另外，为了解决溢出问题，我们希望考虑在分解过程中动态伸缩列，以使数字保持在一定范围内。与深度学习中的信道量化一样，为每一列分配一个不同的范围也是一种可行的方法。

2.8 对称特征值问题

在[35]中描述了一种算法，用于提高给定特征值及其相伴的特征向量的准确性。该算法具有类似于求解线性系统的迭代改进的良好性质。这种方法假设一个特征对是给定的。对于特征对是如何找到的没有任何假设，是通过对物理问题的一些知识，是一个较低精度的初始特征值分解，还是一个巧妙的猜测。

我们对提高特征对的准确性非常感兴趣。考虑特征值问题 $Ax = \lambda x$ ，其中 λ 和 x 已通过某种方式被得到。由于这个特征对是通过计算机上有限精度的计算或对问题的深入了解而得到的，因此它们一般不是真正的特征值和特征向量，而是一种近似值。然而，我们知道存在 μ 和 $\tilde{y}$ ，使得 $A(x + \tilde{y}) = (\lambda + \mu)(x + \tilde{y})$ 是特征值问题的精确解，其中 μ 和 $\tilde{y}$ 分别是对计算值 λ 和 x 的修正。

我们将 x 标准化，使得 $\|x\| = 1$ ，且称 $x_s = 1$ ，其中 x 的第 s 个分量是所有分量中最大的。这是可以做到的，因为我们在选择 x 的分量时有一个自由度。我们假设 x 的第 s 个分量是精确的，不需要修正。这确定了 $\tilde{y}_s$ 的值， $\tilde{y}_s$ 是对 x_s 的修正。由于 x_s 是精确的，所以 $\tilde{y}_s = 0$ 。这也通过 x 和 $\tilde{y}$ 之间的关系确定了修正量 $x + \tilde{y}$ 的自由度，即 $x_s + \tilde{y}_s = 1$ 。

我们可以将方程 $A(x + \tilde{y}) = (\lambda + \mu)(x + \tilde{y})$ 转化为 $(A - \lambda I)\tilde{y} - \mu x = \lambda x - Ax + \mu \tilde{y}$ 。注意到 $\lambda x - Ax$ 是近似值的残差。如果我们更仔细地观察乘积 $(A - \lambda I)\tilde{y}$ ，我们会发现，因为 $\tilde{y}_s = 0$ ，所以 $A - \lambda I$ 的第 s 列实际上没有参与乘积 $(A - \lambda I)\tilde{y}$ 。在式子 $(A - \lambda I)\tilde{y} - \mu x$ 中，我们可以将 $\tilde{y}_s$ (它是 0) 用 μ 代替，将 $(A - \lambda I)$ 的第 s 列用 $-x$ 代替，从而得到 $(A - \lambda I)\tilde{y}$ 。

我们令 $y := \tilde{y} + \mu e_s$ ，其中 e_s 是单位矩阵的第 s 列，所以新定义的 y 的第 s 个分量的值为 μ ，即 $y_s = \mu$ ，我们把矩阵 $(A - \lambda I)$ 中的第 s 列用 $-x$ 替换所得到的矩阵定义为 B ，因此我们可以将 $(A - \lambda I)\tilde{y} - \mu x = \lambda x - Ax + \mu \tilde{y}$ 重写为 $By = r + y_s \tilde{y}$ ，其中 $r = \lambda x - Ax$ 。

由于解向量的 $n + 1$ 个元素是已知的，我们将用 B 的截断形式来求解，截断后 $n + 1$ 行和 $n + 1$ 列不再存在。这个截断是可以完成的，因为我们知道解向量在第 $n + 1$ 个位置是零。上述方程是一个定义修正 y 的非线性方程。这个系统可以用下面的迭代方法求解：

$By^{(p+1)} = r + y_s^{(p)} \tilde{y}^{(p)}$ ，其中 $\tilde{y}^p = y_s^{(p)} - y_s^{(p)} e_s$ 。这是 [36] 中使用的方法。

算法7: 对称特征值问题的迭代优化

```

1: Input:  $A = A^T \in \mathbb{R}^{n \times n}$ ,  $\hat{X} \in \mathbb{R}^{n \times \ell}$ ,  $1 \leq \ell \leq n$ 
2: Output:  $X' \in \mathbb{R}^{n \times \ell}$ ,  $\tilde{D} = \text{diag}(\tilde{\lambda}_i) \in \mathbb{R}^{\ell \times \ell}$ ,  $\tilde{E} \in \mathbb{R}^{\ell \times \ell}$ ,  $\omega \in \mathbb{R}$ 
3: function  $[X', \tilde{D}, \tilde{E}, \omega] \leftarrow \text{REFSYEV}(A, \hat{X})$ 
4:    $R \leftarrow \mathbb{I}_n - \hat{X}^T \hat{X}$ 
5:    $S \leftarrow \hat{X}^T A \hat{X}$  6:    $\hat{\lambda}_i \leftarrow s_{ij} / (1 - r_{ii})$  for  $i = 1, \dots, \ell$   $\triangleright$  计算近似特征值
7:    $\tilde{D} \leftarrow \text{diag}(\tilde{\lambda}_i)$ 
8:    $\omega \leftarrow 2 \left( \|S - \tilde{D}\|_2 + \|A\|_2 \|R\|_2 \right)$ 
9:    $e_{ij} \leftarrow \begin{cases} \frac{s_{ij} + \tilde{\lambda}_j r_{ij}}{\tilde{\lambda}_j - \tilde{\lambda}_i} & \text{if } |\tilde{\lambda}_i - \tilde{\lambda}_j| > \omega \\ r_{ij}/2 & \text{otherwise} \end{cases}$  for  $1 \leq i, j \leq \ell$   $\triangleright$  计算更优矩阵 $\tilde{E}$ 的元素
10:   $X' \leftarrow \hat{X} + \hat{X} \tilde{E}$ 
11: end function

```

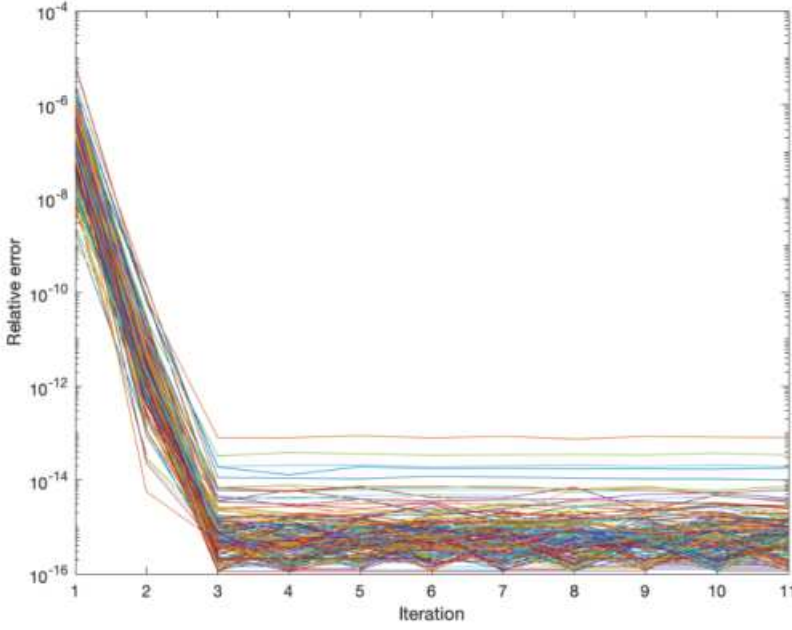


图 8 $n = 200$ 时特征值从单精度细化到双精度的收敛性

算法7展示了另一种方法，使用迭代求精过程来求解对称特征值问题[37]。该方法对于聚类特征值[38]也是适用的。第4、5和10行是该算法中运算的密集部分，相当于对矩阵间的乘法函数 $xGEMM$ 的4次调用。第8行是矩阵范数。第9行是构造细化矩阵的元素运算。第10行是应用细化矩阵对特征向量的更新。除了第8行的矩阵范数外，所有的计

算都需要高精度的算法。尽管该算法只能对特征向量的一个子集起作用，但它只能在相应的子空间中进行细化。因此，细化过程可能是有限的。换句话说，如果只对光谱的一部分进行更高精度的细化，那么期望的精度可能是无法达到的。

图8显示了在细化整个特征谱时算法7在大小为 $n = 200$ 的实对称矩阵上的收敛行为。每条线表示一个特征值的收敛性，并且标准化残差 $\|Ax - \lambda x\|/\|A\|\|x\|$ 根据后续迭代次数绘制。

基于线性解的迭代细化也是可能的[36]。算法8是称为*SICE*的过程，它在每次迭代中，求解由秩1更新产生的线性系统，以细化单个特征对。在替换 $A - \lambda I$ 中的一列时引入秩1更新，以去除特征向量修正的一个自由度，同时计算相应特征值的修正。原始公式[36]使用两个系列的Givens 旋转求解系统，使其成为上三角形。这个过程很难在现代架构上并行化。此外，在使用该算法细化多个特征值时，应该考虑某种形式的正交化。

在许多应用中，我们满足于特征值特征向量对的一个子集。在这种情况下，一旦矩阵被简化为三对角形式，使用诸如Multiple Relatively Robust Representations(MRRR)[39]之类的算法来计算特征对会更有效。虽然这种方法本身的精度不如同类方法(Divide 、Conquer 和QR)，但是[40]表明使用混合精度方法可以有利于提高求解的精度和总的求解时间。这里的混合精度方法在正交性方面要优于其单精度和其他求解算法。

3 多精度算法的数据和通信压缩

通过使用多精度来加速科学计算的一个基本要求是能够在所采用的不同浮点格式之间有效地转换数据，并最小化与这些数据移动相关的通信。有效地完成此任务的技术和实现通常是专门开发的，例如，针对使用混合精度的特定算法和实现进行实践和调整。因此，尽管有一些解决方案可以解决特定的问题，但是没有标准，通常没有用户级别的接口可以用于较低级别的构建块，因此没有可以提取为其他开发人员使用的，独立的、受支持的库或组件基础设施。为了解决这个问题，我们一直在研究一些构建模块，这些模块可以被提取并包含在数值库中，用于开发混合精度算法。对数据和通信压缩有帮助的组件将在后面的小节中讨论。

算法8:迭代地精确计算特征值的算法

```
1: function  $[x, \lambda] \leftarrow \text{SICE}(A, x_0, \lambda_0)$ 
2:  $[Q, T] \leftarrow \text{schur}(A)$   $\triangleright$  利用Schur分解找到  $A = QTQ^T$  其中T 是准上三角的.
3:  $[m, s] \leftarrow \max(x_0)$   $\triangleright$  在特征向量中找到最大值和索引;
4:  $x_0 \leftarrow x_0/m$   $\triangleright$  标准化;
5: for  $i = 1, 2, \dots$  do
6:  $c \leftarrow -x_{i-1} - (A - \lambda_{i-1}I)[:, s]$   $\triangleright A - \lambda_{i-1}I$  的第s列
7:  $d \leftarrow Q^T c$ 
8:  $f \leftarrow e_s^T Q$   $\triangleright Q$  的第s 行
9: 求解秩1更新系统  $Q(T - \lambda_{i-1}I + df^T) Q^T y_i = Ax_{i-1} - \lambda_{i-1}x_{i-1}$ 
10:  $\lambda_i \leftarrow \lambda_{i-1} + y_i[s]$   $\triangleright$  特征值校正
11:  $x_i \leftarrow x_{i-1} + y_i$   $\triangleright$  特征向量校正
12:  $x_i[s] \leftarrow x_{i-1}[s]$   $\triangleright$  还原  $x[s]$ 
13: if  $2 \times y_i[s] > y_{i-1}[s]$  then .
14: break from for loop
15: end if
16: end for
17:  $x \leftarrow x_i$ 
18:  $\lambda \leftarrow \lambda_i$ 
19: end function
```

3.1 数据转换

许多混合精度算法需要在不同的IEEE标准格式之间转换数据。例如，LAPACK根据其混合精度迭代细化求解器的需要支持这种类型的数据转换。根据标准LAPACK命名约定和矩阵表示，通过辅助例程为一般矩阵或三角矩阵提供支持。例如，一般的矩阵可以从FP64转换到FP32，如下所示：

$$\text{zlag2c}(M, N, \text{zA}, \text{LDA}, \text{cA}, \text{LDCA}, \text{INFO})$$
$$\text{dlag2s}(M, N, \text{dA}, \text{LDA}, \text{sA}, \text{LDSA}, \text{INFO})$$

第一个例子用于将双复矩阵转换为单复矩阵，第二个例子用于双复矩阵到一个单实数矩阵。另一种方式(从单到双)也提供了Clag2z和slag2d转换过程。

转换三角矩阵的接口如下:

```
zlat2c(UPLO, N, zA, LDA, cA, LDCA, INFO )
```

```
dlat2s(UPLO, N, dA, LDA, sA, LDSA, INFO )
```

从单到双的接口分别是clat2z和slat2z。

这些例程遵循LAPACK的接口，在MAGMA中也为GPU被提供。MAGMA 还为一般矩阵增加了从单精度到半精度(FP32到FP16)的转换:

```
slag 2 h(M, N, sA, LDA, hA, LDHA, INFO, QUEUE)
```

以及对应的hlag2s.这些例程对NVIDIA GPU进行了很好的优化支持AMD GPU (通过hipMAGMA)。MAGMA还提供了批量的等价物用于批量转换.

一个更专业的混合精度计算库可能必须支持一个更完整的数据转换例程，例如数组、跨数组、张量等，以及更多的组合格式，包括用户/应用程序定义的。例如，一些混合精度FFT算法(见第3.3节)使用高精度阵列(如FP32)的动态“分裂”为两个低精度(例如，FP16)阵列。关于格式的进一步讨论和扩展请参见第7 节。.

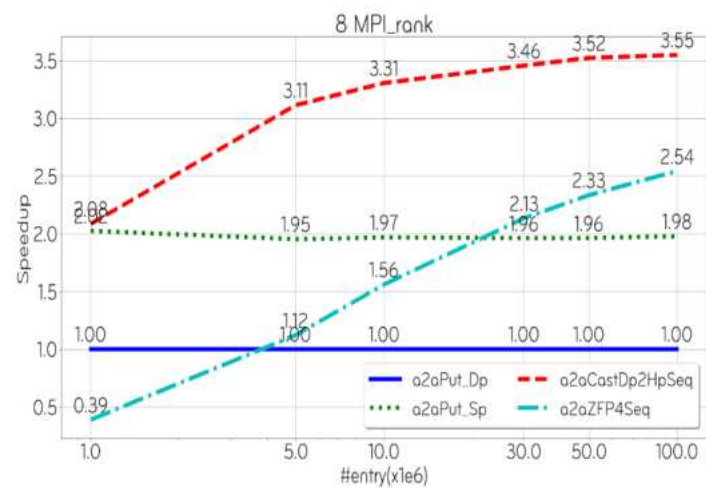


图 9 在Nvidia V100 GPU上，All2All使用4×压缩(红色)，使用2×压缩(点绿色)以及使用ZFP 4×压缩的加速对比

3.2 数据压缩

用于减少通信的数据压缩是开发混合精度算法所需的另一个组成部分。我们所考虑的最简单的形式是类型转换，如3.1节所述。这是有损压缩的一个例子。例如，从FP64到

FP32的铸造直接导致大约8位小数精度的损失，但减少了2倍的数据大小。使用混合精度迭代优化技术，Casting已经将MAGMA 中的FP64求解器加速到4倍[24,4,3]，我们将其作为基准来评估使用其他压缩算法的潜力。

我们评估了使用ZFP 压缩的可能性。ZFP 提供了有损数据压缩算法，用户可以指定压缩模式为固定速率、固定精度[41]。ZFP 在压缩浮点数据时引入的舍入误差分析见[42]。本实验中的值采用随机数，指定的压缩是4。注意，与casting相比，压缩率与casting to fp16 相同，但精度与casting to fp32 相当。这些结果使得我们使用像ZFP 这样的工具来加速内存绑定代码(例如FFT (参见3.3节))达到4 是可行的，同时失去了大约8个十进制数的准确度。

3.2.1 混合精度MPI

令人感兴趣的是MPI 扩展，它将随后的数据转换与MPI 通信融合在一起。转换必须是用户指定的，包括转换或其他数据压缩或转换机制，其中单个MPI 调用将按照指定的方式转换输入数据，发送转换后的数据，相应的MPI 调用将按照用户指定的方式接收并再次转换结果。我们的MPI 合作者已经为All2All 和P2P 通信开发了初步的混合精度MPI。结果表明，对于足够大的数据，MPI 通信可以按照数据压缩的比例加速，即转换可以忽略不计。这些实现适用于CPU，也适用于使用GPU导向通信的GPU。

我们的初步结果也可以使用ZFP 来压缩数据。图9显示了fp64中All2All 的加速结果(标记为基，即1行的加速)。

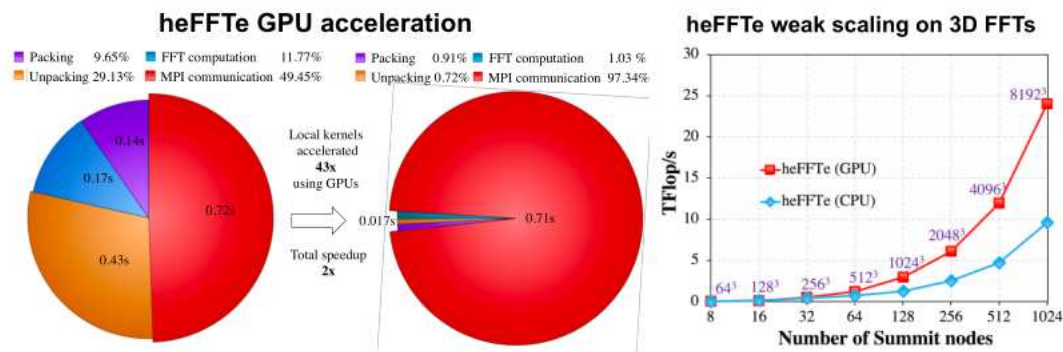


图 10 左:在4个Summit节点上使用gpu与10243 FFT的cpu加速heFFTe的对比。值得注意的是，使用gpu可以加速节点计算43倍。右:显示heFFTe 的弱可伸缩性，图中显示最多1024个节点(6v100GPU; 双复数算法; 伴随程序块开始和结束; 性能假设为 $5N^3 \log_2 N^3$ 个浮点)

请注意，这里我们使用了ZFP，并设法将MPI通信加速到2倍以上，而只丢失了大约8位十进制数字的精度，即我们实现了一个比使用转换到FP32的相应版本的性能更好的加速(同时具有相同的精度)。数据发送本身在被压缩4x时加速接近4x，但是当增加数据压缩和解压缩的成本时，整体加速会下降。这意味着理论上加速仍然可以达到4x的实现，GPU工作在压缩和解压缩是流水线的，并可与通信重叠。

3.3 近似快速傅里叶变换

本节描述的混合精度技术的一个应用是通过混合精度算法来加速多维FFT。我们发现超过12个ECP 应用程序在其代码中使用FFTs，例如LAMMPS、HACC、exam 和Copa 共同设计的应用程序[43]。对于需要基于FFT 的求解器的ECP 应用程序来说，缺乏针对分布式异构并行系统的快速且可伸缩的3d FFT 历程，正如对即将到来的百万兆等级运算系统所预测的那样，一些应用程序已经表示有兴趣探索使用近似FFT，以牺牲一些精度损失来提高性能。

为了满足大规模GPU 系统上对高性能可伸缩FFTs 的上述需求，我们开发并发布了针对Exascale (heFFTe)库的高效FFTs [44,45,46]。heFFTe v0.2具有非常好的可伸缩性和性能，接近峰值值的90% (参见图10)。然而，在使用GPU (vs CPU)加速了大约43个本地/节点的计算之后，主要的瓶颈变成了MPI 通信。目前，在经典的FFT 问题上，GPU 只能在大约1% 的时间内使用，也就是说，GPU 99% 的时间可以免费用于其他计算，而其余99% 的时间用于MPI 通信。因此，MPI 通信中的任何加速都会转化为整个FFT 计算的同等加速。

3.3.1 权衡精度-速度的近似FFT

使用混合精度来加速FFT的一个想法是通过对速度和精度进行权衡。在这里，我们通过压缩数据来减少通信量，例如，通过铸造或压缩，如第3.2节所述。

多维FFTs 使用all2all 类型的MPI 通信，其中第一个数据被打包(在每个GPU 上本地打包，使用并受益于GPU 的高带宽)，接下来是MPI 通信，最后在每个GPU 上本地解压数据[43,44]。由于打包和拆包受内存限制，需要一次性处理数据，因此可以将铸造或其他类型的压缩与打包/拆包融合在一起，从而大大减少消耗。这个想法是通过使用混合精度MPI 来探索的，它融合了所有这些操作(如第3.2.1节)。获得的加速度的量化与准确性的降低如图9所示。

目前的工作是将GPU在压缩/解压缩上的使用与MPI 通信通过流水线重叠。这是可能的，因为FFTs 是内存限制的，GPU 可以在99% 的时间内自由地进行其他计算，例如，作为Summit 硬件配置的基准测试。

3.3.2 精度控制

精度要求主要依赖于应用程序，可以通过指定所需的压缩速率来控制。由此产生的混合精度FFT 在保持相同性能的前提下，在存储和计算精度“低”的情况下，其精度要高于相应的FFT。此外，在极端情况下，可以调整混合精度FFT使其具有相同的精度，例如FP64 fft。例如，可以通过计算本地计算中由于舍入造成的十进制数字损失来实现这一点，并使用适当的数据压缩速率，以便只有有效的数字才能进行通信。

3.3.3 动态拆分

另一种加速多维FFT 的思想是将高精度FFT 数据动态分解为两个比例尺的低精度FFT 数据集，并行地对两个数据集进行FFT 变换，最后将结果进行组合。这是在fp32数据被分成两个fp16集的背景下进行的探索，以便对fp16数据应用快速的GPU Tensor core 计算[47,48]，例如，通过一个小基数，例如，4，其中FFT 矩阵可以精确地表示(即使在fp16 中)，并受益于Tensor core 加速的HGEMMs。该思想的一个扩展可以考虑到FFT 矩阵除了一个小的基数矩阵以外不需要组合，以便将其应用于数据作为GEMM。因此，没有太多的计算负担(由于FFT 的内存限制性)，人们可以使用更高精度的傅里叶矩阵和计算(而不是存储数据的精度)来加速整个FFT 计算。

4 多精度稀疏分解

稀疏系统的直接方法也可以从使用低精度格式中获益。其想法是利用硬件提供的更快的速度，以较低的精度执行高消耗计算。然后，使用一些更便宜的“修复”算法以更高的精度恢复精度。稀疏分解，如稀疏LU 和QR 分解，最常用于构造稀疏直接求解器。这里有两个相关但正交的研究方向。第一个是关于因式分解本身，第二个是关于直接求解器的。

4.1 多精度稀疏LU和QR

与密集LU 和QR 因式分解类似，计算的很大一部分在于整个消除步骤中的shur三角化。在密集的情况下，shur三角化的大部分工作可以通过GEMM 操作来实现。然而，在稀疏情况下，每个舒尔三角化通常遵循三个步骤: 1)将稀疏数据结构中的值收集到连续内存中，2)执行GEMM 操作，3)将GEMM 输出分散到目标稀疏数据结构中。

使用较低精度的好处是双重的: 对于步骤2)，我们可以使用供应商提供的非常快的较低精度的GEMM 函数，例如那些利用NVIDIA 的Tensor core 的函数。对于步骤1和步骤3中的gather/s-catter，数据移动量将会减少。

对于密集的情况，主要的优势来自于加速后的GEMM 速度。但是在稀疏情况下，GEMM 只是上面三个步骤的一部分。此外，GEMM 内核调用的维度通常较小，并且在整个分解过程中大小不一致。因此，仅从GEMM 获得的速度增益是有限的。我们将需要设计新的方案来增强GEMM 计算与gather/catter操作的搭接。

4.2 多精度稀疏直接求解器

对于密集情况，在第2.2节中我们重新讨论了混合精度迭代细化(IR)算法，该算法根据前一次迭代收敛的结果进行自适应精度调整。算法可以提供高精度的解决方案，即使昂贵的LU 和QR 分解是以较低的精度完成的。我们回忆在算法9中使用三个精度的IR 算法[49,50]。这个算法已经可用在LAPACK 的xGERFSX 函数中。

使用了以下三种精度:

- ε_w 是用于存储输入数据a 和b 的工作精度。它是求解器中使用的最低精度，也是输出所需要的精度。
- ε_x 是储存计算结果 $x^{(i)}$ 的精度，要求 $\varepsilon_x \leq \varepsilon_w$ ，如果需要分量收敛，可能需要 $\varepsilon_x \leq \varepsilon_w^2$ 。
- ε_r 是计算残差 $r^{(i)}$ 的精度，一般有 $\varepsilon_x \leq \varepsilon_w$ ，且通常最少是工作精度的两倍 $\varepsilon_r \leq \varepsilon_w^2$ 。

算法9:直接求解器的三精度迭代求解

- 1: 使用基本解法(例如LU 或QR)求解 $Ax^{(1)} = b \quad \triangleright (\varepsilon_w)$
 - 2: $i=1$
 - 3: **repeat**
 - 4: $r^{(i)} \leftarrow b - Ax^{(i)} \quad \triangleright (\varepsilon_r)$
 - 5: 使用基本解法(例如LU 或QR)求解 $Adx^{(i+1)} = r^{(i)} \quad \triangleright (\varepsilon_w)$
 - 6: 更新 $x^{(i+1)} \leftarrow x^{(i)} + dx^{(i+1)} \quad \triangleright (\varepsilon_x)$
 - 7: $i \leftarrow i + 1$
 - 8: 直到 $x^{(i)}$ “足够精确” 为止
 - 9: 返回 $x^{(i)}$ 和误差界
-

利用上述步骤和 ε_x 和 ε_r 的自适应调整,如果A的范数条件数不超过 $1/(\gamma(n)\varepsilon_w)$,则该算法以较小的范数误差和误差界收敛.类似的，如果A的分量的范数条件数不超过 $1/(\gamma(n)\varepsilon_w)$,则该算法以较小的分量范数误差和误差界收敛.此外，这个IR程序可以返回给用户可靠的范数和分量范数的误差边界。[49]中的误差分析都应该贯彻到稀疏情况。

以下是精度的配置示例:

- $\varepsilon_w = 2^{-53}$ (IEEE-754 双精度), $\varepsilon_x = 2^{-53}$, $\varepsilon_r = 2^{-106}$ (双-双)

- $\varepsilon_w = 2^{-16}$ (B-float), $\varepsilon_x = 2^{-24}$, $\varepsilon_r = 2^{-53}$

我们的计划是首先将上述算法扩展到稀疏直接求解器SuperLU 和STRUMPACK。我们将解决以下开放性问题:

- 当 ε_w 是bfloat16,误差分析和误差范围可能需要重新考虑.
- 稀疏LU/QR(第1、5行)和稀疏matvec(第4行)的相对工作量与稠密的相对工作量是不同的。对于典型的三维偏微分方程离散化问题,其工作量分别为 $O(n^2)$ 和 $O(n^{4/3})$. 因此,“昂贵”与“廉价”之间的比率比稠密情况要小,我们需要注重更高精度的计算。

5 Krylov求解器技术的多精度工作

我们回顾的范围包括基于lanczos的(短期递归)和基于Arnoldi的(长期递归)方法以及求解线性方程组 $Ax = b$ 的相关方法。在长期递归方法的背景下,我们考虑Arnoldi - QR算法和改进的广义最小残差(GMRES) Krylov子空间方法迭代求解线性方程组。这里的重点是研究迄今为止采用的方法,将这些方法将混合精度浮点运算结合起来,以加快计算速度,同时在完全双精度运算中保留原始算法的部分或全部数值特性(即表示误差和正交性损失)。

5.1 Lanczos-CG

5.1.1 理论结果

我们首先非常简要地总结了Lanczos和CG方法的有限精度计算中最著名的结果,并讨论了该领域的现有进展以及如何将这些结果扩展到混合精度情况。我们注意到有大量关于基于lanczos方法的有限精度计算的文献,我们不在这里完全描述这些文献。为了更全面的叙述和历史参考,我们建议读者阅读Meurant和Strako的手稿[51]。

Chris Paige 推导了有限精度Lanczos 中不具正交性和其他重要性质的基本关系[52]。这些结果随后被Anne Greenbaum 用来证明CG方法[53]的后向稳定性结果;即,格林鲍姆表明,有限精度的CG 可以看作是在更大的线性系统上运行的精确CG,其中系数矩阵的特征值包含在原矩阵的特征值周围的紧簇中(这些簇的直径取决于矩阵的性质和机器的精度)。Greenbaum还在她称之为“递归计算残差方法”的有限精度中证明了最大可达精度的基本结果,其中包括CG、BICG、BICGSTAB和其他基于lanczos的方法[54]。Paige和Greenbaum的结果也被推广到[55]的s阶Lanczos/CG变体,结果表明,有限精度下的s 阶Lanczos 与经典Lanczos 一样具有较低的“有效”精度(这种“有效”精度取决于用于生成s 阶基的多项式的条件)。我们相信这些现有的结果可以推广到混合精度的情况,在Paige 的分析[52]中,他首先定义了一个 ε_0 用于表示内积中的误差和一

个 ε_1 用于表示矩阵向量积中的误, 然后这些量可以在后来的定理中结合起来以简化分析。我们可以扩展他的分析, 把这两个量分开; 这些结果也可以在格林鲍姆[53]的框架下进行解释。

基于lanczos的混合精度方法的现有成果包含在“不精确Krylov子空间方法”的工作中, 也适用于基于arnoldi的方法; 可以参考西蒙西尼和Szyld[56], van den Eshof和Sleijpen[57]的手稿。在这样的框架内, 假设矩阵-向量乘积计算伴随一些有界扰动(在每次迭代中可能会发生变化), 所有其他的计算都是精确的。这些方法的动机是提高应用程序的性能, 在这些应用程序中, 矩阵向量乘积占计算成本的主导地位, 例如, 当矩阵密集或应用涉及求解一个线性系统时。关于“非精确Krylov子空间方法”的许多理论结果在文献中得到了证实, 这些结果大多集中在可获得的最大精度上。一个令人惊讶的结果是, 在不影响最大可达精度的情况下, 矩阵-向量乘积中的不精确性可以随着迭代的进展以与残差范数的倒数成比例的速度在范数中增长。然而, 一个关键的实际问题是, 在达到可达到的精度之前, 不精确性是否会影响收敛行为; 这在像CG这样的短期复发方法中是完全可能的, 而且在理论上还没有得到很好的研究。

5.1.2 实际应用

我们简要地提到在实际应用中使用混合精度克里洛夫子空间方法的工作, 侧重于性能而不是理论结果。

Clark et al.[58]的工作就是一个例子, 它使用混合精度CG和BICGSTAB方法, 在一个运行在gpu上的Lattice QCD应用程序中实现了Sleijpen和van der Vorst[59]的“可靠更新”策略。“可靠更新”策略背后的思想是计算真实残差, 并在选择的迭代中用来替换递归更新残差, 从而提高可达到的精度; 这与解向量的批量更新一起完成。通过仅在真实残差计算和分组更新中使用更高的(双精度)精度(以及在其余的计算中使用单精度或半精度), 作者声称他们能够实现全双精度。这值得进一步的理论研究, 我们认为可以将[59]的结果以及van der Vorst和Ye[60]的相关工作推广到混合精度设置中。

5.2 Arnoldi-QR MGS-GMRES

对于MGS-GMRES, Gratton 等人的混合精度工作。Al. [61]是最新且最优的, 特别是由Bjorck [62]和Paige [52]推导出的正交性损失的关系, 后来由Paige、rozloznik 和strakos [63]改进, 用于提供混合精度计算的公差。当Krylov向量的线性独立性丧失时, MGS-GMRES的收敛停滞(范数方向相对向后误差趋于 ε), 这是由Paige的S矩阵范数 $\|S\|_2 = 1$ 所表现出来的。S矩阵[64]是由Giraud等人[65]在舍入误差分析中出现的下三角T矩阵推导出来的。

综上所述, Gratton等人[61]的论文假设从Arnoldi-QR算法出发, 采用改进的Gram-Schmidt算法, 并在MGS-GMRES迭代求解器中采用精确算法。Arnoldi-QR算法应用于非对称矩阵A时产生了矩阵分解, 失去了正交性 F_k :

$$AV_k = V_{k+1}H_k, \quad V_{k+1}^T V_{k+1} = I + F_k \quad (0.0.1)$$

接下来, 他们引入了不精确(例如单精度)的内积——这直接与MGS产生的 $A = QR$ 分解的正交关系丢失有关。由此产生的正交性损失, 由 $|I - Q^T Q|_2$ 所测量的, 增长为 $\mathcal{O}(\varepsilon)\kappa(A)$, 由Björck[62]得到, 使用Paige, Rozložník 和Strakoš [66, 63] 及相关工作设计的Arnoldi-QR, 增长为 $\mathcal{O}(\varepsilon)\kappa([r_0, AV_k])$ 。不精确的内积为:

$$h_{ij} = v_i^T w_j + \eta_{ij}$$

其中 h_{ij} 是Hessenberg矩阵 H_k 的元素, Arnoldi-QR算法生成矩阵的QR分解: $[r_0, AV_k] = V_{k+1}[\beta e_1, H_k]$ 下面给出了 F_k 的正交性缺失关系, 其中矩阵U是严格上三角形

$$F_k = \bar{U}_k + \bar{U}_k^T, \quad U_k = \begin{bmatrix} v_1^T v_2 & \cdots & v_1^T v_{k+1} \\ & \ddots & \\ & & v_k^T v_{k+1} \end{bmatrix}$$

定义矩阵

$$N_k = \begin{bmatrix} \eta_{11} & \cdots & \eta_{1k} \\ & \ddots & \\ & & \eta_{kk} \end{bmatrix}, \quad R_k = \begin{bmatrix} h_{21} & \cdots & h_{2k} \\ & \ddots & \\ & & h_{k+1,k} \end{bmatrix}$$

Björck[62]推导出的正交性缺失的关系, 可以通过将改进的Gram-Schmidt算法进行的 $A = QR$ 分解应用到Arnoldi-QR算法中得到

$$N_k = -[0, U_k] H_k = -U_k R_k$$

MGS-GMRES中Krylov向量完全丧失正交性(线性独立性)时标志着误差达到最小, 当范数相对向后误差达到 $\mathcal{O}(\varepsilon)$ 时, GMRES停止或不再前进, Gratton et al. [61]展示了如何保持足够的正交性以达到所需的残差精度, 通过在一定的公差上将内积从双精度转换为单精度, 将其与van den Eshof和Sleijpen[57]以及Simoncini和Szyld[56]中的不精确矩阵向量乘积结合起来。

在实践中, 重新开始的GMRES 算法通常用于减少内存需求。该算法同时产生隐式和显式残差。因此, 我们可能会问, 是否可以以降低精度的方式执行。本文所描述的混

合精度的迭代细化工作可以应用于分析重新开始的GMRES (m)的收敛性, 假设迭代次数是固定的, 因为重新开始的GMRES 只是以GMRES 作为修正项的求解器的迭代细化。但是, 还没有进行更详细的实验分析。我们相当肯定, 残差计算必须以更高的精度执行, 以实现范数向后误差接近双精度机器舍入。

5.3 替代方法

尽管有些超出了本文的范围, 但我们可以证明, 基于Świrydowicz等人[67]提出的MGS算法的逆紧凑WY形式, 可以修改Gratton等人的分析[61]。与其同等对待MGS-GMRES算法中的所有内积, 不如从失去正交关系的角度考虑严格上三角矩阵 $U = L^T$ 。我们引入单精度 $L_{:,j-1} = Q_{j-1}^T q_{j-1}$ 和双精度三角形来求解 $r = (I + L_{j-1})^{-1} Q_{j-1}^T a$ 来更新R, 这将直接采用highlam [68]的正向误差分析。前者影响正交性的缺失, 而后者影响QR的表示误差-同样影响Arnoldi-QR, 但可以以单一精度计算更多(或大部分)的内积。

图11展示了正交性的变化, 在双精度下用标准MGS (蓝色)下的内积和 $\|I - Q^T Q\|$ 计算 $A = QR$ 较在单精度下(在MATLAB 中模拟)用逆紧WY MGS (红)和 $Q_{j-1}^T q_{j-1}$, 我们能够观察到至少一致或者更高的误差水平。X 轴是随机生成矩阵的对数条件数。下三角形的解是以双精度计算的。

Barlow [69]包含了类似的结果, 如果不是相同的块形式的算法公式。他的工作与Björck 1994年的论文相关联[70, 第7节], 该论文使用MGS的递归形式导出三角矩阵T, 在文献中被称为“紧WY”表示。Björck 使用了一个下三角矩阵作为MGS的紧WY形式, Malard和Paige[71]导出了Barlow也用过的上三角形式, 它颠倒了投影的顺序。后者是不稳定的, 因为向后递归导致 $\mathcal{O}(\varepsilon)\kappa^2(A)$ 失去正交性。Julien Langou 的一个有趣的观察结果是, 上三角形比下三角形更不稳定(即使向后-向前算法导致重新正交化; 参见Leon, Björck, Gander [72]中的算法)。

Barlow[69]参考了Chiara Puglisi[73]的工作并采用Householder逆紧型WY表示, Joffrain等人[74]讨论过——这被称为Householder的“逆紧WY”表示最初来自Walker对Householder GMRES 的研究[75]。然后, Barlow 将这种方法扩展到块状紧型MGS; 参见Sun的报告[76]。

Świrydowicz等人[67]的贡献是注意到MGS存在一种反紧凑的WY表示——拥有投影

$$P^{IM} = I - Q_{j-1} T^{IM} Q_{j-1}^T = I - Q_{j-1} (I + L_{j-1})^{-1} Q_{j-1}^T$$

并“放缩”范数 $\|q_{j-1}\|_2$, 使得这些可以在一次全局化简中计算出来。Barlow[69]将块形式(实际上, 这在他的方程(3.10)中给出)和文献Puglisi[73] 建立了这种联系。Björck和

Paige[77]根据Sheffield 的观察得出Householder和MGS之间的联系。Paige将其定义为augmented, Gratton 等[61]也引用了这个结果。Paige最近也把这些改进的想法扩展到Lanczos。T矩阵出现在Paige和W. Mulling的研究中[78], 后来在[64]中推导出了正交性缺失度矩阵 $S = (I + L_{j-1}^T)^{-1}L_{j-1}^T$, 这也出现在Giraud、Gratton和Langou的文章中[65]; Langou还与Barlow和Smoktunowicz[79]一起研究了勾氏技巧, 以减少计算向量范数和减少计算经典Gram-Schmidt (CGS)类cholesky形式时出现的错误误差。

为了在MGS-GMRES中合并单双浮点运算, 起初我们似乎可以以单精度存储T矩阵, 但随后我们仍然必须形成 $Q_{j+1}^T a$, 并以双精度存储 Q_{j+1} . 通过进一步成本权衡, 我们可以使用一种基于前向求解器递归的重新正交化形式

$$T = (I + L_{j-1}^T)^{-1}(I + L_{j-1})^{-1}$$

初步的计算结果表明, 这种方法是可行的, 可以将相对残差以双精度趋近到 $\mathcal{O}(\epsilon)$, 而在单精度上保持到 $\mathcal{O}(\epsilon)$ 的正交性。

用MGS计算得到的 $A + E = QR$ 的表示误差(向后误差)不受单精度内积的影响, 仍然为 $\mathcal{O}(\epsilon)$ 。我们不知道之前是否已经知道这一点。

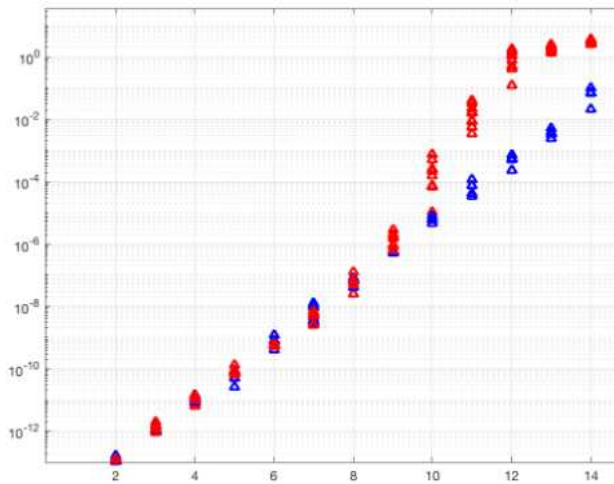


图 11 混合单双MGS算法的正交性损失

6 多精度预处理器

在大型稀疏系统的迭代求解过程中, 预处理器是促进令人满意收敛的重要组成部分。预处理的概念是将病态线性系统 $Ax = b$ 转变为(左)预处理系统 $MAx = Mb$ ($AMy =$

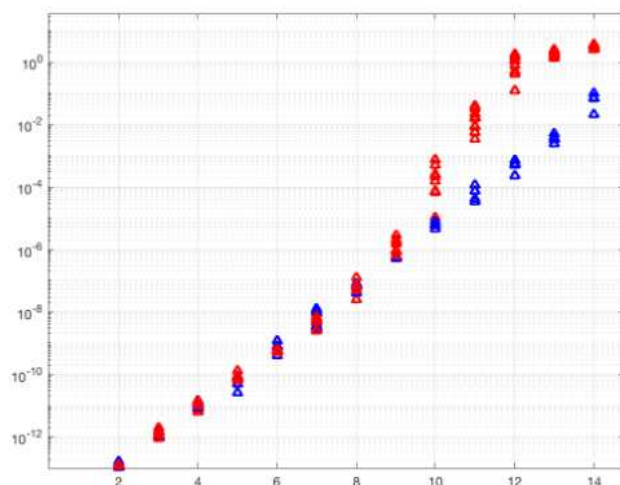


图 12 矩阵的GMRES残差与正交性损失 $\|S\|_2$

b , $x = My$ 用于右预处理), 这使得迭代求解器更快地收敛。收敛特性通常取决于目标系统的条件。对于病态的 A , 也要求预处理器是病态的。否则, 不能期望预处理器改善问题的条件或迭代解的收敛性。在这方面, 预处理器本质上尽力在逼近系统矩阵的逆矩阵。显然, 如果预处理器得到的是精确逆, 那么解是容易获得。然而, 计算精确逆的成本高得令人望而却步, 并且在大多数情况下, 预处理器只是系统矩阵逆的粗略近似。因此, 很自然地会质疑对本质上仅具有有限精度的预处理器是否需要使用高精度。实际上, 只要使用较低精度的格式导致的精度损失既不影响预处理器的精度, 也不影响其规律性, 那么为预处理器选择较低精度格式是一种有效的办法。例如, Trilinos允许在高精度迭代求解器中使用低精度预处理器, 见第9节, hypre团队致力于在低精度下运行第一个循环的多重网格方法。但是, 在预处理器应用程序中使用较低精度会导致与使用高精度时不同的舍入效果。具体来说, 舍入效应使预处理器变成变量, 因为舍入效应不仅大于高精度, 而且还取决于输入数据[80]。因此, 低精度预处理器只能用于加速可以处理变量预处理器的迭代方法, 也就是说即使预处理器在两次迭代之间发生变化也可以收敛。对于生成与先前搜索方向正交的搜索方向的Krylov子空间求解器, 变化的预处理器需要将预条件搜索方向与先前的预条件搜索方向进行额外的正交化。灵活的Krylov求解器(例如FGMRES、FCG)包含这种额外的正交化, 所以会稍微贵一些。同时, 它们确实可以使用低精度的预处理器, 这可以补偿额外的成本。

另一种解决方法是将内存精度与算术精度解耦, 参见第7节, 并且仅以低精度存储预处理器, 但以高精度应用预处理器[80]。以高精度运行所有算术使预处理器保持不变, 并且无需对预条件搜索方向进行额外的正交化。另一方面, 将内存精度与算术精度解耦

需要在从主存储器读取数据时在两种格式之间进行动态转换。幸运的是，大多数迭代求解器和预处理器都受内存限制，并且转换可以隐藏在内存传输之后。Ginkgo库中提供了自适应精度块-Jacobi预处理器，可以实现将内存精度与算术精度解耦，请参见第9节。

7 将算术格式与内存格式解耦的多精度工作

7.1 在多重网格方法中使用不同的精度格式

在整个硬件技术链中，我们看到计算能力（以每秒浮点运算计算）与通信能力（以内存带宽计算）之间的差距越来越大。在现代处理器技术中，从主存储器中检索值比执行算术运算花费的时间要长几个数量级，并且集群的不同节点之间的通信也比主存储器访问慢几个数量级。结果，越来越多的算法撞上了内存墙——如今，ECP生态系统中的几乎所有应用程序都受到现代硬件架构的内存限制。在没有颠覆性硬件更改的情况下，我们面临着所有应用程序与主存储器或节点之间通信缓慢的情况。

克服这个问题的一个有希望的且也许是唯一有希望的策略是更谨慎地利用带宽容量，减少通信量和通信点的数量，并尽可能地利用计算来交换通信。具体来说，其思想是从根本上将内存精度与算术精度解耦，仅在计算中采用高精度，并在访问主存储器中的数据或与远程处理器通信时，尽可能降低精度[81]。在这种情况下，一个重要方面是“内存访问器”的设计，它可以在IEEE 高精度算术格式和内存/通信格式之间实时转换数据，请参见图13。此外，内存/通信格式不一定必须是IEEE 标准的一部分，但也可以是符号、指数和有效位的任意组合[82]，甚至是非标准格式如L.Gustafson 的Unum格式[83]。显然，这与使用压缩技术补充格式解耦的想法有着密切的联系，如第3节中提出的那样。

将算术精度与内存精度解耦的想法来自Ginkgo稀疏线性代数库中可用的自适应精度块-Jacobi预处理器[80]，这里的想法是计算一个高精度的块-Jacobi预处理器，然后以最低浮点精度格式存储不同的逆对角块，以避免溢出并仍然保持预处理器的规则性，见图14。

这种存储格式是为每个对角线块单独选择的，分别反映了其特征。图15（顶部）可视化了在存储稀疏矩阵集的对称正定矩阵的24阶的逆对角块时的格式分布。显然，转换为较低格式通常会降低线性算子的准确性，但由于块-Jacobi预处理器忽略所有非（块）对角线元素，它们通常只是矩阵逆的粗略近似，于是设计上只有非常有限的准确性，实验结果表明，在大多数情况下，使用较低精度的格式存储逆对角块对预处理器的有效性和外部求解器的收敛性的影响可以忽略不计。

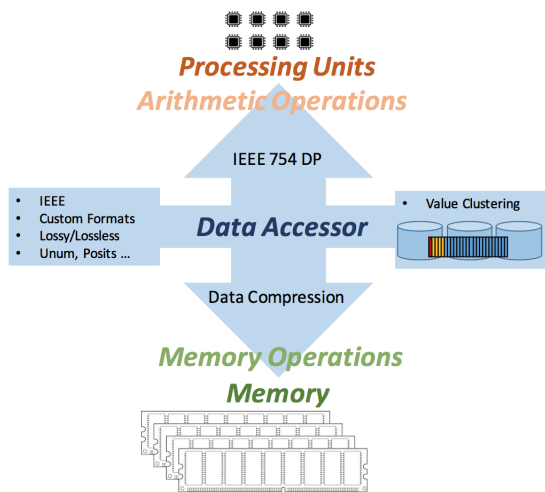


图 13 访问器将内存格式与算术格式解耦，并在每次内存访问中实现动态数据转换。

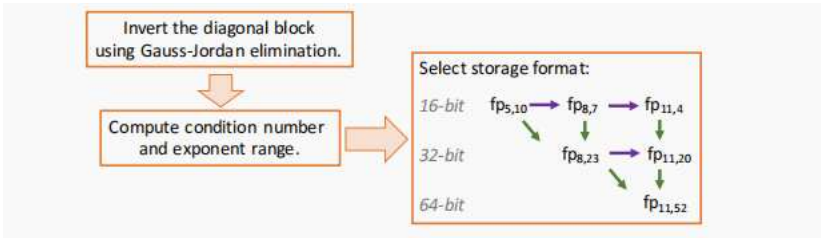
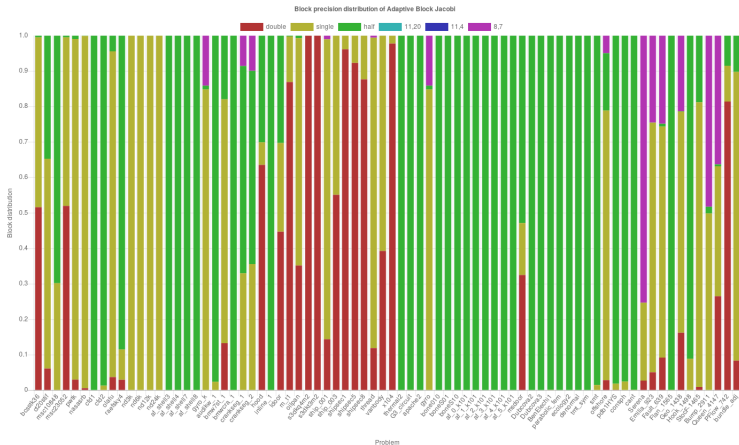


图 14 块-Jacobi的存储格式优化：从最紧凑的存储（左上角)开始，格式以指数位扩展以适应数据范围(向右)并保持规律性(向下)，直到两者都满足。



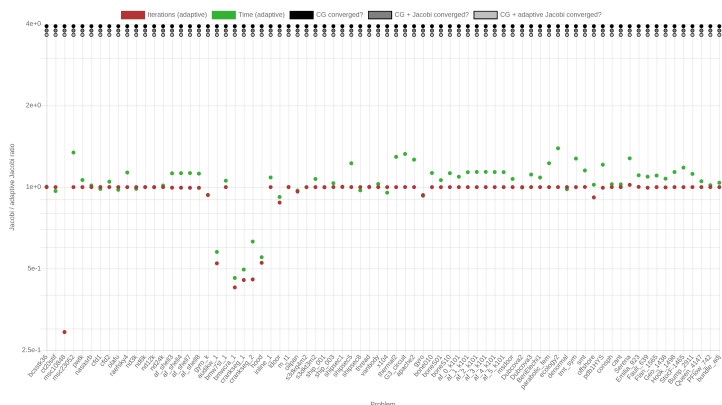


图 15 顶部：当保留每个反向对角块的1位精度时，不同块之间的浮点格式分布。每列代表Suite Sparse Matrix Collection的一个对称正定矩阵。底部：对解决系统引起的线性问题的顶级CG求解器的影响，对于大多数系统，收敛速度不受使用较低存储精度格式的影响。所有预处理器应用程序都更快，从而平均减少20% 的运行时间。

同时，以较低精度存储逆对角块会减少每个预处理器应用程序中的内存访问量，从而加速带宽受限的迭代求解过程，参见图15。对于自适应精度块-Jacobi预处理器，重要的是访问器将逆对角线块转换回IEEE标准精度,不仅是出于性能原因——利用处理器的高度优化的IEEE浮点运算，而且也是出于数值原因——在预处理器应用程序的算术运算中使用工作精度可以保留预条件作为常数运算符，以较低精度应用预处理器将导致变量预处理器，并且需要使用(更昂贵的)灵活的迭代求解器[80]。

多重网格方法是非常有效的迭代方法。基本上有两种类型的多重网格方法：几何多栅方法（GMG）和代数多栅方法（AMG）。GMG要求每一层都有实际的网格来生成其组件，而AMG可以被认为更像一个黑盒方法，因为它可以被赋予一个矩阵，并将使用合理的启发式方法自动生成每一级的组件。这些方法是多精度处理的一个有趣的目标，因为它们的不同组件以不同方式影响整个算法。GMG 和AMG组件将平滑器、粗网格、限制和延长运算符结合在一起。此外，研究不同级别的精度变化也很有意义。最后，GMG和AMG可以用作其他求解器的预处理程序，也就是说，在整个预处理程序中使用较低的精度的可能性。历史上，大多数工作集中在使用较低精度的GMG或AMG方法作为双精度求解器的预处理。此外，还有人试图在MG中应用ZFP[41]或为AMG 建立一个误差分析框架。

Ljungkvist 和Kronbichler[84,85]采用的是无矩阵几何多重网格方法成功地使用混合精度来解决不同阶数的拉普拉斯问题。他们的求解器基础结构允许使用混合精度算术，以单精度执行多重网格V型循环，并以双精度进行外部修正，最高可增加83%。

同样, Glimberg等人[86]使用单精度多重网格对双精度缺陷校正方案进行预处理, 并在GPU架构上解决非线性水波应用中的拉普拉斯问题。与双精度版本相比, 混合精度版本的加速比高达1.6, 纯单精度版本的加速比高达1.9。

Yamagishi和Matumura[87]也在他们的非静力学海洋模型中对Poisson/Helmholtz问题采用了单精度多网格和双精度的锥度梯度求解器。他们报告说, 单精度Matvec比双精度Matvec的速度提高了2倍, 并且使用这种方法提高了总体时间, 但是他们只将完整的应用程序运行与其中央处理器版本进行了比较。

有各种出版物追求实用单精度AMG 预处理程序到双精度求解器的相同策略。

Emans和van de Meer[88]对多核处理器上预处理Krylov 求解器的单个核进行了仔细分析, 包括稀疏矩阵-向量乘法 (SpMV), 它构成了AMG的很大一部分。他们还考虑了通信的影响, 其中较低的精度导致较小的信息量, 但延迟仍然是一个问题, 特别是在AMG的最粗糙的层面。他们发现, 使用混合精度的预调节器几乎不影响收敛性, 因此内核的加速 (在1.1和1.5 之间) 有可能延续到整个求解器, 并导致计算流体力学应用程序中运行时间的改进。

Sumiyoshi等人[89]针对各向同性和各向异性泊松问题, 研究了具有中央处理器和图形处理器的异构计算机体系结构上的AMG性能。他们认为平滑聚合AMG是一个独立的求解器。他们仔细分析了五个不同的架构上算法的不同部分, 包括一个多核CPU集群。他们报告说, 与双精度版本相比, 混合精度实现的GPU-CPU架构的加速比在1.2到1.6之间。这些加速与这些架构上的SpMV性能有关 (1.6 到1.8之间)。然而, 混合精度版本在只使用中央处理器的架构上稍慢, 这对SpMV操作上几乎没有取得任何改进。

Richter等人[90]研究了应用于双精度PCG求解器的单精度AMG求解器 (ML和PET Sc) 的性能。他们将该方法用于图形处理器/中央处理器计算机架构上的高压隔离器的静电模拟。他们的混合精度版本需要的时间大约是双精度版本的84%。

Kate Clark[91]在一篇演讲中描述的方法将混合精度的使用进一步扩展到半精度。Clark和他的合作者使用双精度缺陷校正方法, 使用单精度Krylov求解器和半精度AMG预处理程序, 取得了良好的效果。

Fox和Kolasinski[92]的另一项有趣的相关研究考察了ZFP (一种有损压缩算法) 在多重网格中的应用。由于ZFP的局部结构, ZFP 可以很容易地被集成到数值模拟中, 而不需要改变底层算法。然而, 由于ZFP是一种有损算法, 它将引入一些误差, 因此, 重要的是要了解由ZFP引起的误差是否压倒了其他传统的误差源, 如离散化误差。研究表明, 对于泊松问题上的MG, 将ZFP应用于近似向量, 可以显著减少内存使用, 从而有望减少运行时间, 同时产生的误差保持在离散化误差以下。由于ZFP的硬件版本尚未问世, 因此无法进行实际运行, 但结果显示了使用GMG和/或AMG作为预处理程序的良

好潜力。

目前, Tamstorf等人[93]似乎是唯一研究多精度多重网格方法理论的人。他们的初衷是为了改善迪士尼电影中布的运动外观, 这需要高于FP64的精度。然而, 他们的理论同样适用于降低精度。他们创建了一个为求解由离散化线性椭圆偏微分方程(PDEs)产生的代数方程的混合精度多重网格版本提供了严格的证明的理论框架。出现的稀疏和对称正定的矩阵确保能够使用所谓的能量或A范数来建立收敛和误差估计。作为矩阵条件数的函数, 发展并分析了多重网格收敛行为的界。理论和数值结果都证实, 使用混合精度版本的V-循环和完全多重网格可以达到离散化精度水平的收敛。这个框架受到Carson和Higham[15]的结果的启发, 但最终为许多PDEs提供了更严格的界限。Tamstor等人[94]进一步拓展了他们的理论框架, 包括量化误差。他们通过平衡量化误差、代数误差和描述误差, 使用边界来指导渐进精度多重网格方案中精度水平的选择。他们表明, 虽然迭代细化在残差和更新计算过程中容易受到量化误差的影响, 但在每次迭代中用于计算校正的V周期更有弹性, 并且如果层次结构中的系统矩阵由于量化而变得不确定, 则继续工作。

8 用于机器学习的低精度和多精度技术

现代高性能计算(HPC)硬件继续经历着一个支持各种精度较低的格式来表示浮点数的持续转变, 以提供更高的性能。然而, 可移植性往往不受关注, 因为硬件往往只服务于特定供应商特别感兴趣的一组特定工作负载。这些例子包括英特尔的Cascade Lake矢量神经网络指令(VNNI)和最近宣布的Xe平台的显卡, AMD的Radeon Instinct卡(MI5、MI8、MI25、MI55、MI60)和NVIDIA的Pascal、Volta和Turing系列的计算卡。最后, ARM在其NEON向量单元规范VFP 8.2-A中包括了16位浮点(FP16)。这些加速器遵循两种类型的16位浮点格式规范: 符合IEEE标准的FLOAT16和扩展范围的BFLOAT16。

同时, 一批新的加速器将降低精度的使用提高到一个新的水平, 因为它们针对的是新的机器学习工作负载。这种新硬件包括高通公司的Cloud AI100、HPE的Dot-Product Engine、麻省理工学院的Eysriss[95]、谷歌的TPU[96]、乔治亚理工学院的MAERI[97]、Intel的Deep Learning Boost、Cerebras的CS-1和Facebook的Zion。

一般来说, 机器学习界在评估多精度方面更加积极, 甚至考虑了1位随机梯度下降[98]。机器学习中的典型用例是使用32位算法进行训练, 并在推理任务中使用不同精度。TensorFlow和pyTorch等流行框架是支持推理的量化。量化是存储张量并使用低于浮点位宽的位宽对其进行计算的方法。即使在机器学习框架中, 对量化的支持也仅限于卷

积神经网络或递归神经网络所需的关键功能，并有一些有限的硬件支持。例如，pyTorch 和TensorFlow支持激活和权重的8位量化。这允许在需要额外的2-4倍性能时使用8位进行推断。在训练方面，已经证明16 位的训练对某些任务是足够的[1,101]。最近的Fordon Bell 获奖者证明了低精度的训练也可以用于科学的机器学习任务[102]。

与我们对科学计算感兴趣的例子的深度学习工作类似，包括以较低的精度训练网络并在较高的精度下进行推理[103,104]。训练和推理之间的计算不平衡甚至高于因式分解和随后的迭代细化。另一个区别是，在神经网络训练的背景下，降低精度可能会作为正则化因子被纳入模型中。

9 xSDK数学库的多精度功能和互操作性

9.1 Ginkgo

Ginkgo是一个现代稀疏线性代数库，可以在多核和多核架构上运行[105]。该库的设计是通过将生态系统的可扩展性与使用平台原生语言CUDA (NVIDIA GPUs)、HIP (AMD GPUs) 或OpenMP (Intel/AMD/ARM多核) 的繁重的、特定于架构的内核优化相结合来指导的。软件开发周期通过单元测试、自动配置和安装，Doxygen代码文档，以及持续集成 (CI) 和持续基准测试框架，确保生产质量的代码。

Ginkgo对值类型使用一个静态模板参数，对整数类型使用一个模板参数，允许以不同的精度格式进行编译。支持的标准值类型格式是IEEE 双精度、IEEE单精度、双复合精度和单复合精度。理论上，Ginkgo也可以编译为任何其他（任意）精度格式，但在IEEE 标准之外，硬件和软件方面的支持都非常有限。

除了可以针对不同的精度格式进行编译外，Ginkgo还具有自适应精度块-Jacobi预处理功能，将内存精度与算术精度解耦，并单独优化逆对角块的存储格式。尽管大量利用了先进的多精度技术，但自适应精度块-Jacobi预处理器的数值考虑是完全自动化的，并且对用户来说是隐藏的，用户可以在不降低数值的情况下使用黑盒算法的功能。基于自适应精度块-Jacobi中获得的知识，Ginkgo 目前正在采用存取器概念，从而将内存精度与算术精度分开，参见第7节。

正在考虑整合到Ginkgo中的一项正交多精度技术是基于值聚类的多精度SpMV。

9.2 heFFTe

Exascale(heFFTe)库的高效FFT（快速傅里叶变换）为Exascale 平台提供了快速且强大的多维FFT例程。heFFTe 利用已建立但特别的软件工具，这些工具传统上是应用程序代码的一部分，但没有提取为独立的、受支持的库。这些多维FFT依赖于第三方1D FFT，或来自FFTW，或来自供应商库。

FFT被用于许多领域的应用，包括分子动力学、频谱估计、快速卷积和相关信号调制和无线多媒体应用。例如，分布式三维FFT是分子动力学计算中使用的最重要的核心之一，它的性能会影响应用程序在大型机器上的可扩展性。同样地，第一原理计算的性能在很大程度上取决于FFT的性能求解器。具体来说，对于DOE，我们发现有许多ECP应用在其代码中使用FFT。然而，目前最先进的FFT 库无法在有节点的大型异构机器上扩展，甚至在具有多个高性能GPU（如多个NVIDIA V100 GPU）的一个节点上也无法扩展。为了满足这些需求，heFFTe v0.2 库的发布展示了非常好的弱可扩展性和强可扩展性，以及接近屋顶线理论峰值性能的90%的高性能，这是通过（1）高效利用GPU 的高带宽，（2）尽可能减少全局通信的算法，以及（3）采用GPUDirect技术和MPI优化来实现的。heFFTe提供多精度功能，支持单精度和双精度运算。heFFTe 是一个C++ 库，使用的算法是模板化的，所以其他精度可以很容易的添加。目前的工作是使用混合精度的MPI和压缩来增加混合精度的能力，如第3.2 节所述。

9.3 hypre

hypre是一个高性能预处理程序和求解器的软件库，用于在大规模并行计算机上解决大型稀疏线性方程组。创建hypre 库的主要目的是为用户提供高级并行预处理程序。该库具有针对结构化和非结构化网格问题的并行多重网格求解器。为了便于使用，这些求解器通过hypre的概念性线性系统接口从应用程序代码中访问，该接口允许各种自然问题描述，包括结构化、半结构化和线性代数接口。（半）结构化界面是基于标准矩阵的界面的替代品，给用户提供了一个更自然的线性系统描述手段，并提供了对结构化多网格求解器的访问，从而可以利用附加信息。

9.4 Kokkos Kernels

Kokkos Kernels项目主要关注稀疏/密集线性代数和图形内核的性能可移植的内核。Kokkos Kernels依靠Kokkos 编程模型实现可移植性。稀疏线性代数内核的重点是支持Trilinos和计算科学应用等框架的需求。使用的稀疏线性代数数据结构是一个压缩行存储。Kokkos Kernels 为稀疏矩阵-向量乘法、稀疏矩阵-矩阵乘法、ILU(k) 因式分解、Gauss-Seidel预处理、三角因子产生于直接求解器或不完全因式分解时的三角求解提供内核。所有这些内核都是以矩阵和向量类型为模板的，允许从最初的软件设计中获得多精度支持。Kokkos Kernels还支持团队级BLAS和LAPACK功能的密集线性代数内核。这使得计算科学应用在为加速器编程时可以在“内环”中使用BLAS和LAPACK操作。BLAS和LAPACK功能也是以标量类型为模板，允许多精度使用。Kokkos Kernels还支持图形内核，如distance-1 coloring、distance-2 coloring 和三角形计数内核。

9.5 MAGMA

MAGMA为异构架构提供了LAPACK和大量高度优化的密集和稀疏线性算法(LA)

程序。除了LAPACK, MAGMA中的其他密集LA例程包括BLAS、Batched BLAS和LAPACK, 以及混合精度因子和求解器。MAGMA稀疏组件提供了对稀疏迭代求解器和预处理程序、大量稀疏矩阵格式和转换例程、SpMV/MM和辅助内核的支持。

MAGMA通过提供混合软件, 结合不同硬件组件的不同算法的优势, 解决了异构计算环境的复杂挑战。MAGMA的LA算法专门针对具有GPU的混合多核系统, 从而使应用能够充分利用每个硬件组件提供的力量。MAGMA为线性系统、最小二乘法、特征值问题和正值问题提供求解器。MAGMA库的设计与LAPACK的功能、数据存储和接口相似, 允许科学家将任何依赖线性代数的软件组件无缝移植到异构架构中。MAGMA允许应用程序充分利用当前多/多核CPU和多GPU的异构系统的力量, 在给定的能源约束下, 提供最快的精确求解时间。

MAGMA提供使用LU、Cholesky 或QR分解的混合精度求解器。在低精度开发方面, 迄今为止, MAGMA 的最新版本(v2.5.3) 提供了一个优化的批处理HGEMM内核, 在相对较小的规模上优于供应商的BLAS。它还提供了一个双精度的 $Ax = b$ 的混合精度线性求解器, 同时在LU因子化过程中利用了半精度的优势。混合精度求解器比直接的FP64求解器快4倍, 而且如果矩阵 $\kappa_{inf ty}(A)$ 的条件数达到 10^5 , 则收敛到双精度。

9.6 PETSc

PETSc是一套数据结构和程序, 用于以偏微分方程为模型的科学应用的可扩展解决方案; TAO是一个可扩展的数值优化库。PETSc/TAO 可以很容易地用于用C、C++、Fortran和Python编写的应用程序代码中。

PETSc是用纯C89编写的(最近扩展为支持C99的一部分, 这些部分由更新的微软C编译器支持)。我们一直强调对Fortran 和C标准的最终可移植性。同时, 我们一直保证PETSc编译器也可以完全使用C++, 而且C编译版可以从C++ 编译的代码中使用。这方面最大的麻烦一直是C和C++中复数处理的差异, 需要大量代码来处理这些差异。PETSc 一次只能为单个标量类型和精度建立, 例如实数和四精度。由于C语言不提供模板, 所以管理多个集成精度是很困难的。对于CPU, PETSc支持半精度(仅限ARM)、单精度、双精度、四精度(仅限GNU编译器)。上述内容适用于基于CPU的系统。对于GPU来说, 在较低的精度的情况下, 求解时间可能会有大幅改善, PETSc将使用其GPU 接口, 允许在GPU上运行时以选定的精度进行计算。如果数值在CPU上是双精度的, 那么在传输到GPU上进行计算时, 它们将被转换为单精度值的。当然, 更理想的情况是数据保留在GPU上, 这不需要较多的转换, 除非特别需要, 例如, 病态条件要求一部分计算以更高的精度完成。

9.7 PLASMA

PLASMA (现代体系结构并行线性代数软件)[5]是一个基于现代OpenMP的软件包

，用于解决密集线性代数中的问题。PLASMA 使用现代任务调度技术的提供最先进算法的实现。PLASMA提供了求解线性系统、最小二乘法问题、特征值问题和奇异值问题的程序。PLASMA 是基于OpenMP 及其数据依赖性跟踪和任务调度。PLASMA 库允许科学家轻松地将他们现有的软件组件从LAPACK移植到PLASMA，以利用新的多核架构。PLASMA提供了LAPACK风格的接口，以获得最大的可移植性和兼容性。还提供了一个具有更有效的数据存储的接口，以实现尽可能接近机器的计算峰值的性能。

9.8 SLATE

SLATE是密集线性代数的分布式GPU加速库，旨在替代ScaLAPACK。为此，SLATE提供了并行的基本线性代数子程序(BLAS)、范数、线性系统求解器、最小二乘求解器、奇异值和特征值求解器。它是使用现代的C++编写的，带有兼容的ScaLAPACK和LAPACK的包装器。

SLATE提供使用LU和Cholesky 分解的混合精度求解器。分解以较低的精度进行，然后应用迭代细化将精度提高到更高的精度。该代码是以两种精度为模板的，目前支持单/双和单复/双复。未来的计划包括使用半精度和更强大的GMRES细化机制。

9.9 STRUMPACK

STRUMPACK是一个分布式的GPU 加速库，用于使用秩结构矩阵近似来处理密集和稀疏线性代数，包括分层半可分(HSS)、分层非对角低秩(HODLR)、蝶形和称为块低秩的非分层格式(BLR)。基线稀疏STRUMPACK 是一个多正面稀疏LU直接求解器。稀疏因子中的正面矩阵可以用上述的秩结构格式近似，作为有效的稀疏预处理器，在触发器和存储器中具有接近最优的复杂度。稀疏STRUMPACK 依赖于用HODLR和蝶形格式的ButtYPACK，并为ButtYPACK 的Fortran库提供C++接口。

STRUMPACK是用现代C++编写的，具有模板化的数据类型来支持各种精度，包括实数和复数、单精度和双精度。它还可以支持半精度。目前，迭代细化和GMRES的工作进度与因子分解相同。

9.10 SuperLU

SuperLU是一种分布式、GPU 加速的稀疏线性系统直接求解器，在LU分解和三角求解中使用超节点技术。它使用MPI+OpenMP+CUDA支持各种形式的并行。还提供了一些程序用来平衡系统，计算条件数，计算相对后向误差，并估计精确解的误差范围。

SuperLU是用C语言编写的，可以从C或Fortran程序中调用。代码库使用宏来模板化数据类型，因此它可以支持各种精度的混合，包括实数和复数、单精度、双精度和半精度。目前，迭代细化是以与分解相同的工作精度进行的。正在进行的工作是提供更低精度的因式分解和更高精度的迭代细化。

9.11 Trilinos

Trilinos项目是科学计算领域的一个主要软件框架，用于解决大规模、复杂的多物理工程和科学问题。Trilinos是面向对象的，组织成大约60个不同的软件包，每个软件包都有特定的重点。这些软件包包括线性和非线性求解器、预处理器（包括代数多网格）、图形分割器、求解器和优化算法等。用户只需要安装与他们试图解决的问题相关的软件包子集。Trilinos支持MPI+X，其中X可以是CUDA、OpenMP等。（任何Kokkos支持的东西）。

在Trilinos中，标量类型是一个模板参数，通常设置为IEEE 双精度，同时也完全支持IEEE 单精度。用户可以在双精度外求解器中采用单精度编译的预处理程序，但必须考虑数值影响，即可能需要一个灵活的Krylov解算器(FCG /FGMRES)。关于在Belos包中使用混合精度的简要讨论在[106]中给出。除了单倍和双倍之外，还可以使用其他标量类型，然而，这并不常见，而且在显式模板实例化(ETI)构建系统中也不支持。

10 IEEE格式和格式转换

10.1 仿真器

在2008年IEEE浮点运算标准修订版中定义的半精度(fp16)浮点格式，以及最近提出的半精度格式bfloat16，越来越多地出现在GPU 和其他加速器中。虽然对低精度算法的支持主要是出于机器学习应用的考虑，但正如前面几节所讨论的那样，通用的数值算法也可以从中受益。由于适当的硬件并不总是可用的，而且人们可能希望试验尚未在硬件中实现的新算法，因此需要对低精度算术进行软件模拟。在[107]中，Higham和Pranesh 讨论了一种使用高精度和正确性的算法来模拟低精度算法的策略，通过四舍五入误差分析解释了这种模拟的结果。提供了一个MATLAB函数`chop2`，可用于有效地模拟fp16、bfloat16和其他低精度算法，有或没有次正规数的表示，并且可以选择舍入到最近值、定向舍入、随机舍入和有效位中的随机位翻转。感兴趣的读者可参考[107]了解更多详情。

10.2 舍入误差分析

对于问题大小为 n 和单位舍入 u 的问题，数值线性代数中的传统舍入误差分析导致涉及常数 $\gamma_n = nu/(1 - nu)$ 时出现向后误差界限。鉴于大规模且可能低精度的计算，这样的界限可能难以提供任何有用的信息。在[108] 中，Higham和Mary 开发了一种新的概率舍入误差分析，可以应用于广泛的算法。通过使用一个浓度不等式并对舍入误差进行概率假设，他们证明了在几个核心线性代数计算中， γ_n 可以被与概率成比例的松弛常数 $(n \log nu)^{1/2}$ 代替，概率以独立于 n 的量为界。新常数 $\tilde{\gamma}_n$ 随着 n 的增长比 γ_n 慢得多。我

们参考[108]、[109] 以了解更多细节。

致谢

这项工作得到了美国E级计算计划(17-SC-20-SC),美国能源部科学办公室和美国国家核安全管理局的资助。这项工作是在美国能源部的支持下,由劳伦斯-利弗莫尔国家实验室根据DE-AC52-07NA27344合同完成的。

免责声明

这份文件是由美国政府的一个机构赞助的工作报告。美国政府、劳伦斯利弗莫尔国家安全有限责任公司及其任何雇员均不对披露的任何信息、设备、产品或过程的准确性、完整性或实用性作出任何明示或暗示的保证,或承担任何法律责任,或声明其使用不会侵犯私人拥有的权利。本协议中以商号、商标、制造商或其他方式提及任何特定的商业产品、过程或服务,并不一定构成或暗示美国政府或劳伦斯利弗莫尔国家安全的认可、推荐或偏爱。作者在此所表达的观点和意见并不一定代表或反映美国政府或劳伦斯利弗莫尔国家安全有限责任公司的观点和意见,且不得用于广告或产品代言。

桑迪亚国家实验室是由桑迪亚国家技术和工程解决方案有限责任公司管理和运营的多任务实验室,桑迪亚公司是霍尼韦尔国际公司的全资子公司,根据合同DE-NA-0003525为美国能源部的国家核安全局服务。

参考文献

- [1] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep Learning with Limited Numerical Precision. In Proceedings of the 32Nd International Conference on International Conference on Machine Learning - Volume 37, ICML' 15, pages 1737 – 1746. JMLR.org, 2015.
- [2] IEEE Standard for Floating-Point Arithmetic. IEEE Std 754-2008, pages 1 – 70, Aug 2008.
- [3] Azzam Haidar, Stanimire Tomov, Jack Dongarra, and Nicholas J. Higham. Harnessing GPU Tensor Cores for Fast FP16 Arithmetic to Speed Up Mixed-precision Iterative Refinement Solvers. In Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, SC ' 18, pages 47:1 – 47:11, Piscataway, NJ, USA, 2018. IEEE Press.
- [4] Azzam Haidar, Panruo Wu, Stanimire Tomov, and Jack Dongarra. Investigating half precision arithmetic to accelerate dense linear system solvers. In Proceedings of the 8th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems, pages 1 – 8, 2017.
- [5] Emmanuel Agullo, Jim Demmel, Jack Dongarra, Bilel Hadri, Jakub Kurzak, Julien Langou, Hatem Ltaief, Piotr Luszczek, and Stanimire Tomov. Numerical linear algebra on emerging architectures: The PLASMA and MAGMA projects. Journal of Physics: Conference Series, 180:012037, July 2009.
- [6] Ahmad Abdelfattah, Stanimire Tomov, and Jack J. Dongarra. Fast Batched Matrix Multiplication for Small Sizes Using Half-Precision Arithmetic on GPUs. In 2019 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2019, Rio de Janeiro, Brazil, May 20-24, 2019, pages 111 – 122. IEEE, 2019.
- [7] James W Demmel. Applied numerical linear algebra. 1997. SIAM, Philadelphia.
- [8] James Hardy Wilkinson. Rounding errors in algebraic processes. Courier Corporation, 1994.

- [9] Cleve B Moler. Iterative refinement in floating point. *Journal of the ACM (JACM)*, 14(2):316 – 321, 1967.
- [10] Gilbert W Stewart. *Introduction to matrix computations*. Elsevier, 1973.
- [11] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, second edition, 2002.
- [12] James Demmel, Yozo Hida, William Kahan, Xiaoye S Li, Sonil Mukherjee, and E Jason Riedy. Error bounds from extra-precise iterative refinement. *ACM Transactions on Mathematical Software (TOMS)*, 32(2):325 – 351, 2006.
- [13] Werner Oettli and William Prager. Compatibility of approximate solution of linear equations with given error bounds for coefficients and right-hand sides. *Numerische Mathematik*, 6(1):405 – 409, 1964.
- [14] Erin Carson and Nicholas J Higham. Accelerating the solution of linear systems by iterative refinement in three precisions. *SIAM Journal on Scientific Computing*, 40(2):A817 – A847, 2018.
- [15] Erin Carson and Nicholas J Higham. A new analysis of iterative refinement and its application to accurate solution of ill-conditioned sparse linear systems. *SIAM Journal on Scientific Computing*, 39(6):A2834 – A2856, 2017.
- [16] Youcef Saad and Martin H Schultz. Gmres: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on scientific and statistical computing*, 7(3):856 – 869, 1986.
- [17] Nicholas J. Higham. Error analysis for standard and GMRES-based iterative refinement in two and three-precisions. MIMS EPrint 2019.19, Manchester Institute for Mathematical Sciences, The University of Manchester, November 2019.
- [18] Nicholas J. Higham, Srikara Pranesh, and Mawussi Zounon. Squeezing a matrix into half precision, with an application to solving linear systems. *SIAM J. Sci. Comput.*, 41(4):A2536 – A2551, 2019.
- [19] Nicholas J. Higham and Srikara Pranesh. Exploiting lower precision arithmetic in solving symmetric positive definite linear systems and least squares problems. MIMS EPrint

2019.20, Manchester Institute for Mathematical Sciences, The University of Manchester, November 2019.

- [20] Erin Carson, Nicholas J. Higham, and Srikara Pranesh. Three-precision GMRES-based iterative refinement for least squares problems. MIMS EPrint 2020.5, Manchester Institute for Mathematical Sciences, The University of Manchester, February 2020.
- [21] Joseph M. Elble and Nikolaos V. Sahinidis. Scaling linear optimization problems prior to application of the simplex method. *Comput. Optim. Appl.*, 52(2):345 – 371, 2012.
- [22] E. Anderson, Z. Bai, C. H. Bischof, S. Blackford, J. W. Demmel, J. J. Dongarra, J. J. Du Croz, A. Greenbaum, S. J. Hammarling, A. McKenney, and D. C. Sorensen. *LAPACK Users’ Guide*. Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, third edition, 1999.
- [23] Philip A. Knight, Daniel Ruiz, and Bora Ucar. A symmetry preserving algorithm for matrix scaling. *SIAM J. Matrix Anal. Appl.*, 35(3):931 – 955, 2014.
- [24] Azzam Haidar, Ahmad Abdelfattah, Mawussi Zounon, Panruo Wu, Srikara Pranesh, Stanimire Tomov, and Jack Dongarra. the design of fast and energy-efficient linear solvers: On the potential of half-precision arithmetic and iterative refinement techniques. In Y-ong Shi, Haohuan Fu, Yingjie Tian, Valeria V. Krzhizhanovskaya, Michael Harold Lees, Jack Dongarra, and Peter M. A. Sloot, editors, *Computational Science-ICCS 2018*, pages 586 – 600. Springer International Publishing, Cham, 2018.
- [25] Nicholas J. Higham and G. W. Stewart. Numerical linear algebra in statistical computing. In A. Iserles and M. J. D. Powell, editors, *The State of the Art in Numerical Analysis*, pages 41 – 57. Oxford University Press, 1987.
- [26] Pierre Blanchard, Nicholas J. Higham, Florent Lopez, Theo Mary, and Srikara Pranesh. Mixed precision block fused multiply-add: Error analysis and application to GPU tensor cores. MIMS EPrint 2019.18, Manchester Institute for Mathematical Sciences, The University of Manchester, September 2019.
- [27] Takeshi Fukaya, Ramaseshan Kannan, Yuji Nakatsukasa, Yusaku Yamamoto, and Yuka Yanagisawa. Shifted cholesky qr for computing the qr factorization of ill-conditioned matrices. *SIAM J. Scientific Computing*, 42:A477 – A503, 2020.

- [28] Yusaku Yamamoto, Yuji Nakatsukasa, Yuka Yanagisawa, and Takeshi Fukaya. Round-off error analysis of the CholeskyQR2 algorithm. *Electron. Trans. Numer. Anal.*, 44:306 – 326, 2015.
- [29] Ichitaro Yamazaki, Stanimire Tomov, and Jack Dongarra. Mixed-precision Cholesky QR factorization and its case studies on multicore CPU with multiple GPUs. *SIAM J. Sci. Comput.*, 37(1):C307 – C330, 2015.
- [30] Ichitaro Yamazaki, Stanimire Tomov, and Jack Dongarra. Stability and performance of various singular value QR implementations on multicore CPU with a GPU. *ACM Trans. Math. Software*, 43(2):10:1 – 10:18, September 2016.
- [31] Åke Björck. Iterative refinement of linear least squares solutions I. *BIT Numerical Mathematics*, 7(4):257 – 278, 1967.
- [32] Åke Björck. Iterative refinement and reliable computing. In M. G. Cox and S. J. Hammarling, editors, *Reliable Numerical Computation*, pages 249 – 266. Oxford University Press, 1990.
- [33] Pytorch 1.4.0 documentation: Quantization.
- [34] Tensorflow lite 8-bit quantization specification.
- [35] J. J. Dongarra, C. B. Moler, and J. H. Wilkinson. Improving the accuracy of computed eigenvalues and eigenvectors. *SIAM Journal on Numerical Analysis*, 20(1):23 – 45, 1983.
- [36] Jack Dongarra. Algorithm 589 sicedr: A FORTRAN subroutine for improving the accuracy of computed matrix eigenvalues. *ACM Transactions on Mathematical Software (TOMS)*, 8(4):371 – 375, December 1982.
- [37] T. Ogita and K. Aishima. Iterative refinement for symmetric eigenvalue decomposition. *Japan Journal of Industrial and Applied Mathematics*, 35(3):1007 – 1035, 2018.
- [38] T. Ogita and K. Aishima. Iterative refinement for symmetric eigenvalue decomposition II: clustered eigenvalues. *Japan J. Indust. Appl. Math.*, 36:435 – 459, 2019. <https://doi.org/10.1007/s13160-019-00348-4>.

- [39] Dhillon, Inderjit S. and Parlett, Beresford N. and Vömel, Christof. The Design and Implementation of the MRRR Algorithm. *ACM Trans. Math. Softw.*, 32(4):0098 – 3500, 2006. <https://doi.org/10.1145/1186785.1186788>.
- [40] Petschow, M. and Quintana-Ort, E. S. and Bientinesi, P. Improved Accuracy and Parallelism for MRRR-Based Eigensolvers-A Mixed Precision Approach. *SIAM Journal on Scientific Computing*, 36(2):C240 – C263, 2014. <https://doi.org/10.1137/130911561>.
- [41] P. Lindstrom. Zfp version 0.5.3, April.
- [42] James Diffenderfer, Alyson Fox, Jeffrey Hittinger, Geoffrey Sanders, and Peter Lindstrom. Error analysis of zfp compression for floating-point data, 2018.
- [43] Stanimire Tomov, Azzam Haidar, Alan Ayala, Daniel Schultz, and Jack Dongarra. Design and Implementation for FFT-ECP on Distributed Accelerated Systems. ECP WBS 2.3.3.09 Milestone Report ICL-UT-19-05, 2019-04 2019.
- [44] Stanimire Tomov, Alan Ayala, Azzam Haidar, and Jack Dongarra. FFT-ECP API and High- Performance Library Prototype for 2-D and 3-D FFTs on Large-Scale Heterogeneous Systems with GPUs. ECP WBS 2.3.3.13 Milestone Report FFT-ECP STML13-27, 2020-01 2020. revision 01-2020.
- [45] Alan Ayala, Stanimire Tomov, Xi Luo, Hejer Shaiek, Azzam Haidar, George Bosilca, and Jack Dongarra. Impacts of Multi-GPU MPI Collective Communications on Large FFT Computation. In *Workshop on Exascale MPI (ExaMPI) at SC19, Denver, CO, 2019-11 2019*.
- [46] Hejer Shaiek, Stanimire Tomov, Alan Ayala, Azzam Haidar, and Jack Dongarra. GPUDirect MPI Communications and Optimizations to Accelerate FFTs on Exascale Systems. *Eu- roMPI’ 19 Posters, Zurich, Switzerland, (icl-ut-19-06), 2019-09 2019*.
- [47] A. Sorna, X. Cheng, E. D’ Azevedo, K. Won, and S. Tomov. Optimizing the Fast Fourier Transform Using Mixed Precision on Tensor Core Hardware. In *2018 IEEE 25th International Conference on High Performance Computing Workshops (HiPCW)*, pages 3 – 7, 2018.

- [48] Xiaohu Cheng, Anumeena Soma, Eduardo D’ Azevedo, Kwai Wong, and Stanimire Tomov. Accelerating 2D FFT: Exploit GPU Tensor Cores through Mixed-Precision. 2018-11 2018.
- [49] J. Demmel, Y. Hida, W. Kahan, X.S. Li, S. Mukherjee, and E.J. Riedy. Error bounds from extra-precise iterative refinement. *ACM Trans. Math. Softw.*, 32(2):325 – 351, June 2006.
- [50] J. Demmel, Y. Hida, E.J. Riedy, and X.S. Li. Extra-precise iterative refinement for overdetermined least squares problems. *ACM Transactions on Mathematical Software (TOMS)*, 35(4):28, 2009.
- [51] Gérard Meurant and Zdeněk Strakoš. The Lanczos and conjugate gradient algorithms in finite precision arithmetic. *Acta Numerica*, 15:471 – 542, 2006.
- [52] Christopher C Paige. Accuracy and effectiveness of the Lanczos algorithm for the symmetric eigenproblem. *Lin. Alg. Appl.*, 34:235 – 258, 1980.
- [53] Anne Greenbaum. Behavior of slightly perturbed Lanczos and conjugate-gradient recurrences. *Lin. Alg. Appl.*, 113:7 – 63, 1989.
- [54] Anne Greenbaum. Estimating the attainable accuracy of recursively computed residual methods. *SIAM J. Matrix Anal. Appl.*, 18(3):535 – 551, 1997.
- [55] Erin Claire Carson. Communication-avoiding Krylov subspace methods in theory and practice. PhD thesis, University of California, Berkeley, 2015.
- [56] Valeria Simoncini and Daniel B Szyld. Theory of inexact Krylov subspace methods and applications to scientific computing. *SIAM J. Sci. Comput.*, 25(2):454 – 477, 2003.
- [57] Jasper van den Eshof and Gerard LG Sleijpen. Inexact Krylov subspace methods for linear systems. *SIAM J. Matrix Anal. Appl.*, 26(1):125 – 153, 2004.
- [58] Michael A Clark, Ronald Babich, Kipton Barros, Richard C Brower, and Claudio Rebbi. Solving lattice QCD systems of equations using mixed precision solvers on GPUs. *Computer Physics Communications*, 181(9):1517 – 1528, 2010.

- [59] Gerard LG Sleijpen and Henk A van der Vorst. Reliable updated residuals in hybrid Bi-CG methods. *Computing*, 56(2):141 – 163, 1996.
- [60] Henk A Van Der Vorst and Qiang Ye. Residual replacement strategies for Krylov subspace iterative methods for the convergence of true residuals. *SIAM J. Sci. Comput.*, 22(3):835 – 852, 2000.
- [61] Serge Gratton, Ehouarn Simon, David Titley-Peloquin, and Philippe Toint. Exploiting variable precision in GMRES. *SIAM J. Sci. Comput.* (to appear), 2020.
- [62] Åke Björck. Solving linear least squares problems by Gram-Schmidt orthogonalization. *BIT Numerical Mathematics*, 7(1):1 – 21, 1967.
- [63] Christopher C Paige, Miroslav Rozložník, and Zdeněk Strakoš. Modified gram-schmidt MGS, least squares, and backward stability of MGS-GMRES. *SIAM J. Matrix Anal. Appl.*, 28(1):264 – 284, 2006.
- [64] Christopher C Paige. The effects of loss of orthogonality on large scale numerical computations. In *International Conference on Computational Science and Its Applications*, pages 429 – 439. Springer, 2018.
- [65] Luc Giraud, Serge Gratton, and Julien Langou. A rank-k update procedure for reorthogonalizing the orthogonal factor from modified Gram – Schmidt. *SIAM J. Matrix Anal. Appl.*, 25(4):1163 – 1177, 2004.
- [66] Christopher C Paige and Zdeněk Strakoš. Residual and backward error bounds in minimum residual Krylov subspace methods. *SIAM J. Sci. Comput.*, 23(6):1898 – 1923, 2002.
- [67] K. Świrydowicz, J. Langou, S. Ananthan, U. Yang, and S. J. Thomas. Low synchronization Gram-Schmidt and GMRES algorithms. *Numer. Lin. Alg. Appl.*, 2020.
- [68] Nicholas J Higham. The accuracy of solutions to triangular systems. *SIAM J. Numer. Anal.*, 26(5):1252 – 1265, 1989.
- [69] Jesse L Barlow. Block modified Gram – Schmidt algorithms and their analysis. *SIAM J. Matrix Anal. Appl.*, 40(4):1257 – 1290, 2019.

- [70] Åke Björck. Numerics of Gram-Schmidt orthogonalization. *Lin. Alg. Appl.*, 197:297 – 316, 1994.
- [71] J. Malard and C.C. Paige. Efficiency and scalability of two parallel QR factorization algorithms. In *Proceedings of IEEE Scalable High Performance Computing Conference*, pages 615 – 622. IEEE, 1994.
- [72] Steven J Leon, Åke Björck, and Walter Gander. Gram-Schmidt orthogonalization: 100 years and more. *Numer. Lin. Alg. Appl.*, 20(3):492 – 532, 2013.
- [73] Chiara Puglisi. Modification of the Householder method based on the compact WY representation. *SIAM J. Sci. Stat. Comput.*, 13(3):723 – 726, 1992.
- [74] Thierry Joffrain, Tze Meng Low, Enrique S Quintana-Ortí, Robert van de Geijn, and Field G Van Zee. Accumulating Householder transformations, revisited. *ACM Transactions on Mathematical Software (TOMS)*, 32(2):169 – 179, 2006.
- [75] Homer F Walker. Implementation of the GMRES method using Householder transformations. *SIAM J. Sci. Stat. Comput.*, 9(1):152 – 163, 1988.
- [76] Xiaobai Sun. Aggregations of elementary transformations. Technical Report DUKE-TR-1996-03, Duke University, Durham, NC, 1996.
- [77] Åke Björck and Christopher C Paige. Loss and recapture of orthogonality in the modified Gram-Schmidt algorithm. *SIAM J. Matrix Anal. Appl.*, 13(1):176 – 190, 1992.
- [78] Christopher C Paige and Wolfgang Wüling. Properties of a unitary matrix obtained from a sequence of normalized vectors. *SIAM J. Matrix. Anal. Appl.*, 35(2):526 – 545, 2014.
- [79] Alicja Smoktunowicz, Jesse L Barlow, and Julien Langou. A note on the error analysis of classical Gram – Schmidt. *Numerische Mathematik*, 105(2):299 – 313, 2006.
- [80] Hartwig Anzt, Jack Dongarra, Goran Flegar, Nicholas J Higham, and Enrique S QuintanaOrtí. Adaptive precision in block-jacobi preconditioning for iterative sparse linear system solvers. *Concurrency and Computation: Practice and Experience*, 31(6):e4460, 2019.

- [81] Hartwig Anzt, Goran Flegar, Thomas Grützmacher, and Enrique S Quintana-Ortí. Toward a modular precision ecosystem for high-performance computing. *The International Journal of High Performance Computing Applications*, 33(6):1069 – 1078, 2019.
- [82] Thomas Grützmacher, Terry Cojean, Goran Flegar, Fritz Göbel, and Hartwig Anzt. A customized precision format based on mantissa segmentation for accelerating sparse linear algebra. *Concurrency and Computation: Practice and Experience*, page e5418, 2019.
- [83] J.L. Gustafson. *The End of Error: Unum Computing*. Chapman & Hall/CRC Computational Science. Taylor & Francis, 2015.
- [84] Karl Ljungkvist and Martin Kronbichler. Multigrid for matrix-free finite element computations on graphics processors. Technical report / Department of Information Technology, Uppsala University, 2017.
- [85] Karl Ljungkvist and Martin Kronbichler. Multigrid for matrix-free high-order finite element computations on graphics processors. *ACM Transactions on Parallel Processing*, 2019.
- [86] Stefan Lemvig Glimberg, Allan Peter Engsig-Karup, and Morten G Madsen. A fast gpuaccelerated mixed-precision strategy for fully nonlinear water wave computations. In *Proceedings of ENUMATH 2011*, 2011.
- [87] Takateru Yamagishi and Yoshimasa Matsumura. Gpu acceleration of a non-hydrostatic ocean model with a multigrid poisson/helmholtz solver. *Procedia Computer Science*, 80:16581669, 2016.
- [88] Maximilian Emans and Albert van der Meer. Mixed-precision amg as linear equation solver for definite systems. In *Proceedings of International Conference on Computational Science, ICCS 2010*, volume 1, page 175183, 2012.
- [89] Yuki Sumiyoshi, Akihiro Fujii, Akira Nukada, and Teruo Tanaka. Mixed-precision amg method for many core accelerators. In *EUROMPI/ASIA 14: Proceedings of the 21st European MPI Users’ Group Meeting*, page 127132, 2014.

- [90] Christian Richter, Sebastian Schops, and Markus Clemens. Gpu-accelerated mixed precision algebraic multigrid preconditioners for discrete elliptic field problems. *IEEE Transactions on Magnetics*, 50(2), 2014.
- [91] Kate Clark. Effective use of mixed precision for hpc. Smoky Mountain Conference 2019.
- [92] Alyson Fox and Avary Kolasinski. Error analysis of inline zfp compression for multigrid methods. 2019 Copper Mountain Conference for Multigrid Methods.
- [93] Rasmus Tamstorf, Joseph Benzaken, and Stephen McCormick. Algebraic error analysis for mixed precision multigrid solvers. *SIAM Journal on Scientific Computing*, 2020. submitted.
- [94] Rasmus Tamstorf, Joseph Benzaken, and Stephen McCormick. Discretization-error-accurate mixed precision multigrid solvers. *SIAM Journal on Scientific Computing*, 2020. submitted.
- [95] Yu-Hsin Chen, Joel S. Emer, and Vivienne Sze. Eyeriss v2: A flexible and high-performance accelerator for emerging deep neural networks. *CoRR*, abs/1807.07928, 2018.
- [96] Norman P. Jouppi, Cliff Young, Nishant Patil, David A. Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, Richard C. Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omernick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, and Doe Hyun Yoon. In-datacenter performance analysis of a tensor processing unit. *CoRR*, abs/1704.04760, 2017.

- [97] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects. *ACM SIGPLAN Notices*, 53(2):461 – 475, 2018.
- [98] Frank Seide, Hao Fu, Jasha Droppo, Gang Li, and Dong Yu. 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech dnns. In *Fifteenth Annual Conference of the International Speech Communication Association*, 2014.
- [99] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 265 – 283, 2016.
- [100] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in pytorch. 2017.
- [101] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. Training deep neural networks with low precision multiplications. *arXiv preprint arXiv:1412.7024*, 2014.
- [102] Thorsten Kurth, Sean Treichler, Joshua Romero, Mayur Mudigonda, Nathan Luehr, Everett Phillips, Ankur Mahesh, Michael Matheson, Jack Deslippe, Massimiliano Fatica, et al. Exascale deep learning for climate analytics. In *SC18: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 649 – 660. IEEE, 2018.
- [103] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited numerical precision. *CoRR*, abs/1502.02551, 2015. accessed: 2018-08-01.
- [104] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. Deep learning with limited numerical precision. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1737 – 1746, Lille, France, 2015. PMLR. accessed: 2018-08-01.

- [105] Hartwig Anzt, Erik Boman, Rob Falgout, Pieter Ghysels, Michael Heroux, Xiaoye Li, Lois Curfman McInnes, Richard Tran Mills, Sivasankaran Rajamanickam, Karl Rupp, et al. Preparing sparse solvers for exascale computing. *Philosophical Transactions of the Royal Society A*, 378(2166):20190053, 2020.
- [106] Eric Bavier, Mark Hoemmen, Sivasankaran Rajamanickam, and Heidi Thornquist. Amesos2 and belos: Direct and iterative solvers for large sparse linear systems. *Scientific Programming*, 20:241 – 255, 2012.
- [107] Nicholas J. Higham and Srikara Pranesh. Simulating low precision floating-point arithmetic. *SIAM Journal on Scientific Computing*, 41(5):C585 – C602, 2019.
- [108] Nicholas J. Higham and Theo Mary. A new approach to probabilistic rounding error analysis. *SIAM Journal on Scientific Computing*, 41(5):A2815 – A2835, 2019.
- [109] Nicholas J. Higham and Theo Mary. Sharper probabilistic backward error analysis for basic linear algebra kernels with random data. MIMS EPrint 2020.4, Manchester Institute for Mathematical Sciences, The University of Manchester, January 2020.